

# Visual Recognition using Deep Learning

## Introduction

林彥宇 教授

Yen-Yu Lin, Professor

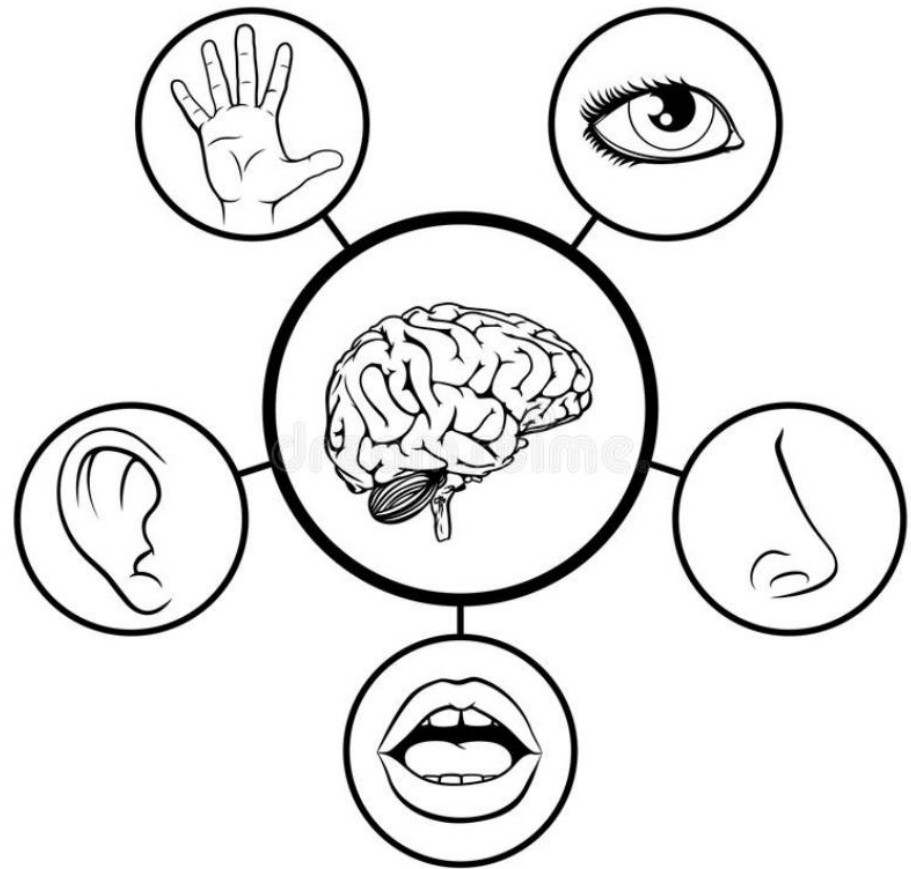
國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Prof. Yung-Yu Chuang

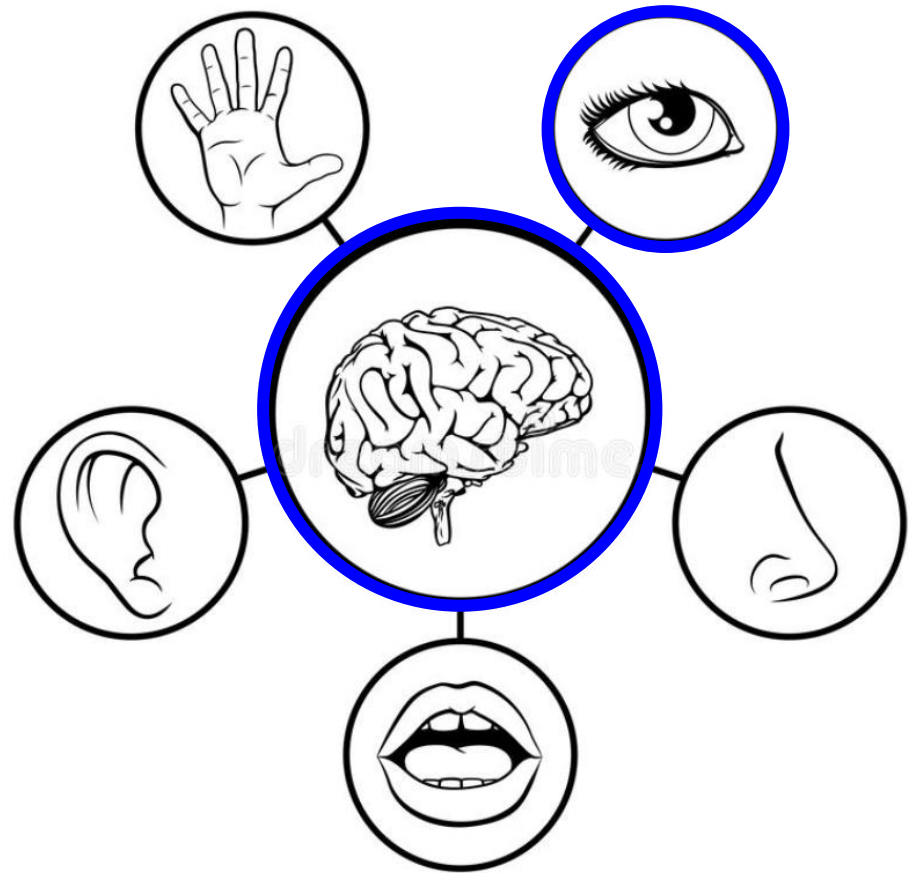
# Artificial intelligence

- Computer vision
- Speech recognition
- Natural language processing
- ....
- Virtual reality
- • ....



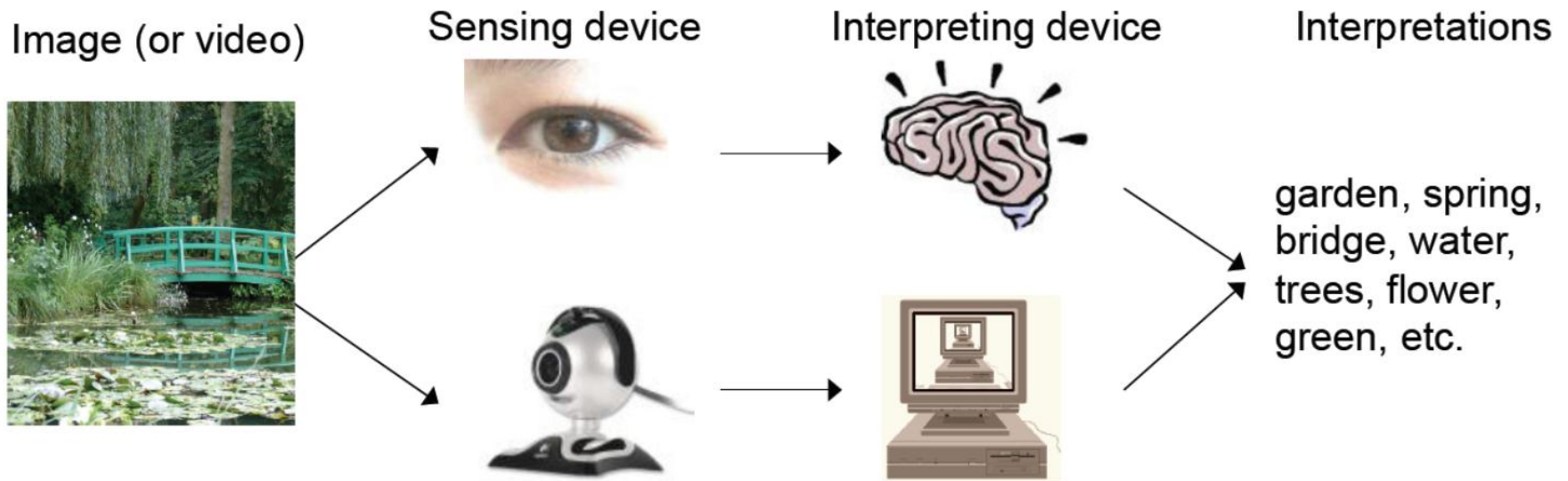
# Artificial intelligence -> Computer Vision

- Computer vision
- Speech recognition
- Natural language processing
- ....
- Virtual reality
- • ....



# Computer vision

- Human vision vs. Computer vision

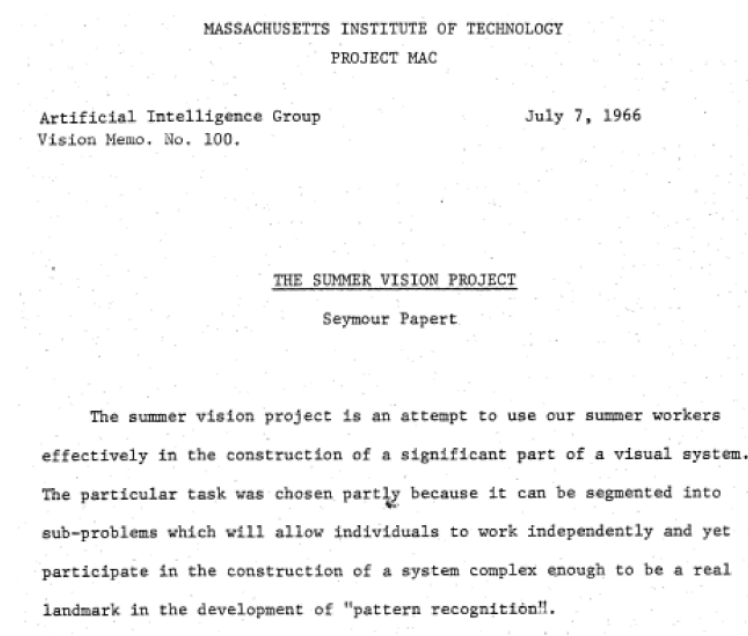


# Is computer vision easy?

- In 1966, Marvin Minsky at MIT asked his undergraduate student Gerald Sussman to “spend the summer linking a camera to a computer and getting the computer to describe what it saw”.



Turing Award, 1969

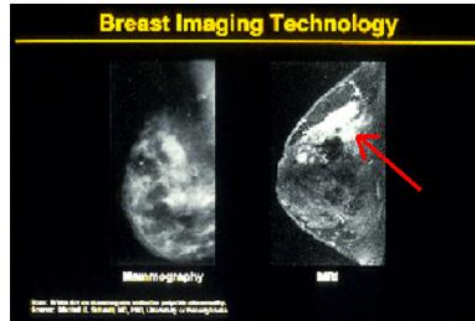


Professor, EE, MIT

# Computer vision applications



Safety



Health



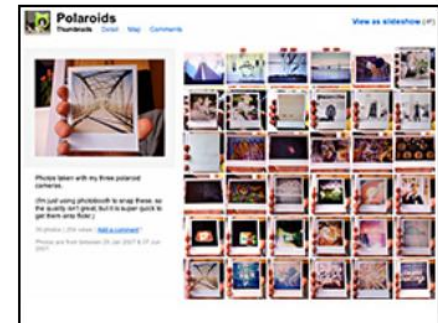
Security



Efficiency



Fun

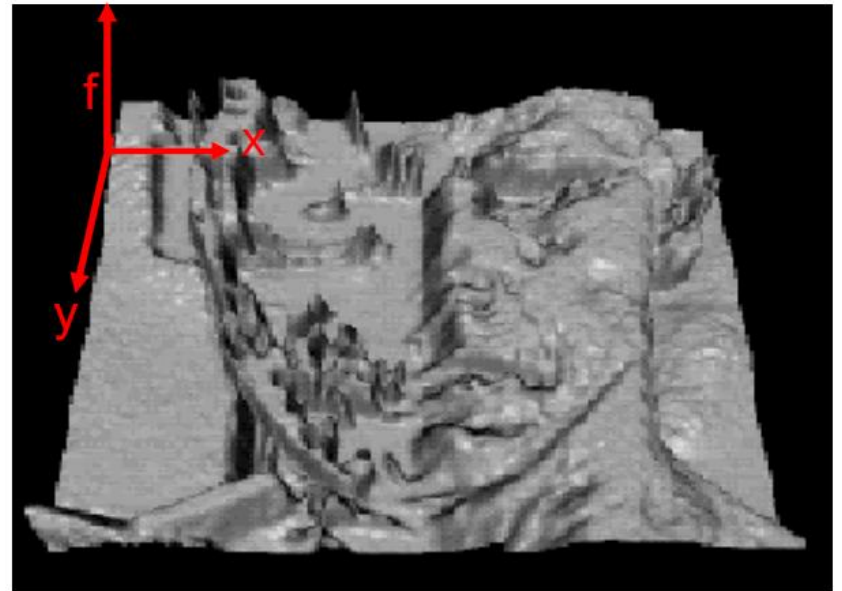


Access



# What is an image?

- We can think of an **image** as a function,  $f: R^2 \rightarrow R$
- $f(x, y)$  gives the **intensity** at position  $(x, y)$



# A digital image

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 243 | 239 | 240 | 225 | 206 | 185 | 188 | 218 | 211 | 206 | 216 | 225 |
| 242 | 239 | 218 | 110 | 67  | 31  | 34  | 152 | 213 | 206 | 208 | 221 |
| 243 | 242 | 123 | 58  | 94  | 82  | 132 | 77  | 108 | 208 | 208 | 215 |
| 235 | 217 | 115 | 212 | 243 | 236 | 247 | 139 | 91  | 209 | 208 | 211 |
| 233 | 208 | 131 | 222 | 219 | 226 | 196 | 114 | 74  | 208 | 213 | 214 |
| 232 | 217 | 131 | 116 | 77  | 150 | 69  | 56  | 52  | 201 | 228 | 223 |
| 232 | 232 | 182 | 186 | 184 | 179 | 159 | 123 | 93  | 232 | 235 | 235 |
| 232 | 236 | 201 | 154 | 216 | 133 | 129 | 81  | 175 | 252 | 241 | 240 |
| 235 | 238 | 230 | 128 | 172 | 138 | 65  | 63  | 234 | 249 | 241 | 245 |
| 237 | 236 | 247 | 143 | 59  | 78  | 10  | 94  | 255 | 248 | 247 | 251 |
| 234 | 237 | 245 | 193 | 55  | 33  | 115 | 144 | 213 | 255 | 253 | 251 |
| 248 | 245 | 161 | 128 | 149 | 109 | 138 | 65  | 47  | 156 | 239 | 255 |
| 190 | 107 | 39  | 102 | 94  | 73  | 114 | 58  | 17  | 7   | 51  | 137 |
| 23  | 32  | 33  | 148 | 168 | 203 | 179 | 43  | 27  | 17  | 12  | 8   |
| 17  | 26  | 12  | 160 | 255 | 255 | 109 | 22  | 26  | 19  | 35  | 24  |



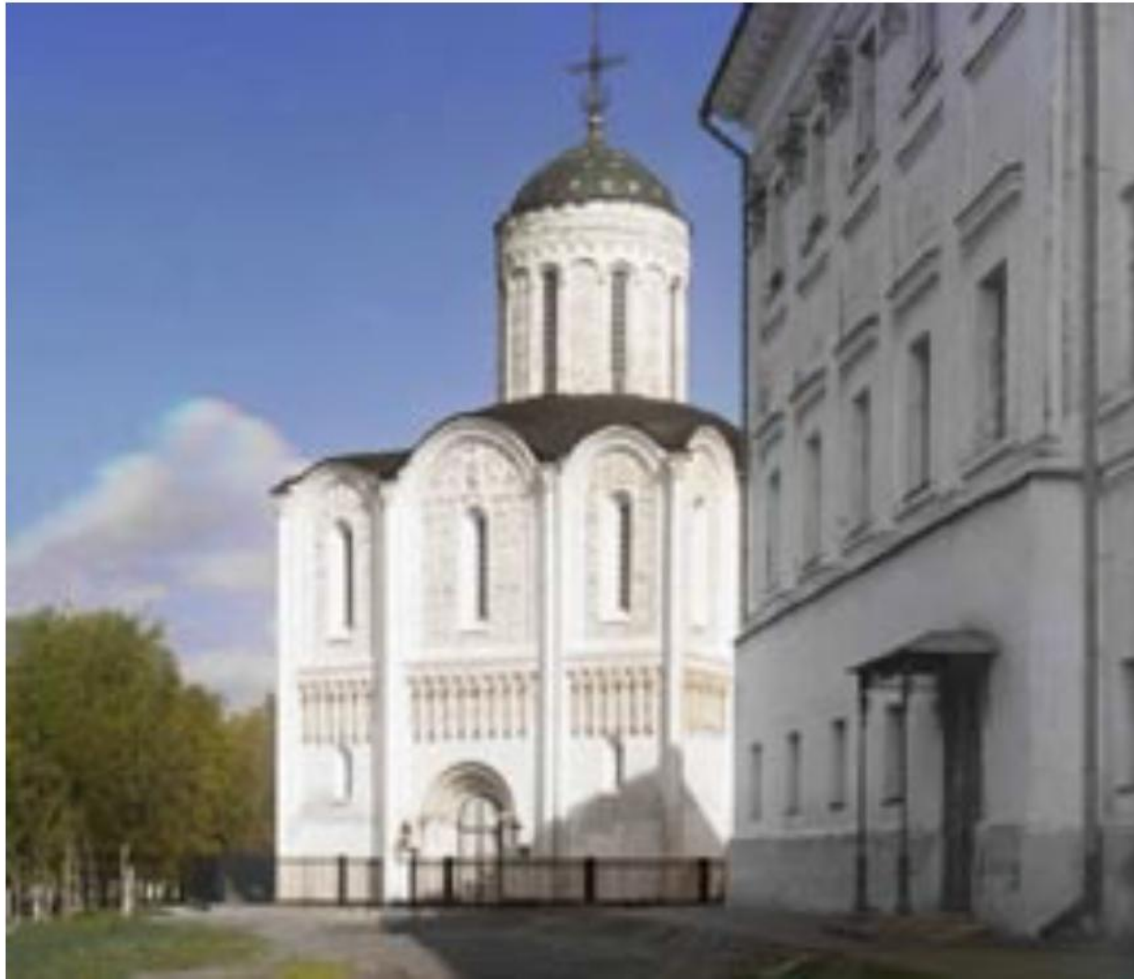


# A digital image

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 243 | 239 | 240 | 225 | 206 | 185 | 188 | 218 | 211 | 206 | 216 | 225 |
| 242 | 239 | 218 | 110 | 67  | 31  | 34  | 152 | 213 | 206 | 208 | 221 |
| 243 | 242 | 123 | 58  | 94  | 82  | 132 | 77  | 108 | 208 | 208 | 215 |
| 235 | 217 | 115 | 212 | 243 | 236 | 247 | 139 | 91  | 209 | 208 | 211 |
| 233 | 208 | 131 | 222 | 219 | 226 | 196 | 114 | 74  | 208 | 213 | 214 |
| 232 | 217 | 131 | 116 | 77  | 150 | 69  | 56  | 52  | 201 | 228 | 223 |
| 232 | 232 | 182 | 186 | 184 | 179 | 159 | 123 | 93  | 232 | 235 | 235 |
| 232 | 236 | 201 | 154 | 216 | 133 | 129 | 81  | 175 | 252 | 241 | 240 |
| 235 | 238 | 230 | 128 | 172 | 138 | 65  | 63  | 234 | 249 | 241 | 245 |
| 237 | 236 | 247 | 143 | 59  | 78  |     | 94  | 255 | 248 | 247 | 251 |
| 234 | 237 | 245 | 193 | 55  | 33  | 115 | 144 | 213 | 255 | 253 | 251 |
| 248 | 245 | 161 | 128 | 149 | 109 | 138 | 65  | 47  | 156 | 239 | 255 |
| 190 | 107 | 39  | 102 | 94  | 73  | 114 | 58  |     |     | 51  | 137 |
| 23  | 32  | 33  | 148 | 168 | 203 | 179 | 43  | 27  | 17  |     |     |
| 17  | 26  |     | 160 | 255 | 255 | 109 | 22  | 26  | 10  | 35  | 24  |



# A color image



# Convolution

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel  
(filter)

=

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

output image

# Convolution

|                |                |                 |   |   |   |
|----------------|----------------|-----------------|---|---|---|
| 3 <sup>1</sup> | 0 <sup>0</sup> | 1 <sup>-1</sup> | 2 | 7 | 4 |
| 1 <sup>1</sup> | 5 <sup>0</sup> | 8 <sup>-1</sup> | 9 | 3 | 1 |
| 2 <sup>1</sup> | 7 <sup>0</sup> | 2 <sup>-1</sup> | 5 | 1 | 3 |
| 0              | 1              | 3               | 1 | 7 | 8 |
| 4              | 2              | 1               | 6 | 2 | 8 |
| 2              | 4              | 5               | 2 | 3 | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel

=

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

output image

# Convolution

|                |                |                 |   |   |   |
|----------------|----------------|-----------------|---|---|---|
| 3 <sup>1</sup> | 0 <sup>0</sup> | 1 <sup>-1</sup> | 2 | 7 | 4 |
| 1 <sup>1</sup> | 5 <sup>0</sup> | 8 <sup>-1</sup> | 9 | 3 | 1 |
| 2 <sup>1</sup> | 7 <sup>0</sup> | 2 <sup>-1</sup> | 5 | 1 | 3 |
| 0              | 1              | 3               | 1 | 7 | 8 |
| 4              | 2              | 1               | 6 | 2 | 8 |
| 2              | 4              | 5               | 2 | 3 | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel

=

|    |  |  |  |
|----|--|--|--|
| -5 |  |  |  |
|    |  |  |  |
|    |  |  |  |
|    |  |  |  |

output image



# Convolution

|   |                |                |                 |   |   |
|---|----------------|----------------|-----------------|---|---|
| 3 | 0 <sup>1</sup> | 1 <sup>0</sup> | 2 <sup>-1</sup> | 7 | 4 |
| 1 | 5 <sup>1</sup> | 8 <sup>0</sup> | 9 <sup>-1</sup> | 3 | 1 |
| 2 | 7 <sup>1</sup> | 2 <sup>0</sup> | 5 <sup>-1</sup> | 1 | 3 |
| 0 | 1              | 3              | 1               | 7 | 8 |
| 4 | 2              | 1              | 6               | 2 | 8 |
| 2 | 4              | 5              | 2               | 3 | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel

=

|    |    |  |  |
|----|----|--|--|
| -5 | -4 |  |  |
|    |    |  |  |
|    |    |  |  |
|    |    |  |  |

output image





# Convolution

|   |   |                |                |                 |   |
|---|---|----------------|----------------|-----------------|---|
| 3 | 0 | 1 <sup>1</sup> | 2 <sup>0</sup> | 7 <sup>-1</sup> | 4 |
| 1 | 5 | 8 <sup>1</sup> | 9 <sup>0</sup> | 3 <sup>-1</sup> | 1 |
| 2 | 7 | 2 <sup>1</sup> | 5 <sup>0</sup> | 1 <sup>-1</sup> | 3 |
| 0 | 1 | 3              | 1              | 7               | 8 |
| 4 | 2 | 1              | 6              | 2               | 8 |
| 2 | 4 | 5              | 2              | 3               | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel

=

|    |    |   |  |
|----|----|---|--|
| -5 | -4 | 0 |  |
|    |    |   |  |
|    |    |   |  |
|    |    |   |  |

output image



# Convolution

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

input image

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

kernel

=

|     |    |    |     |
|-----|----|----|-----|
| -5  | -4 | 0  | 8   |
| -10 | -2 | 2  | 3   |
| 0   | -2 | -4 | -7  |
| -3  | -2 | -3 | -16 |

output image

# Convolution

- One kernel detects one particular pattern in an image
- The following kernel detects vertical edges in an image

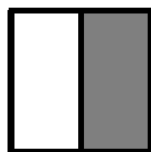
|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |

\*

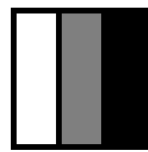
|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

=

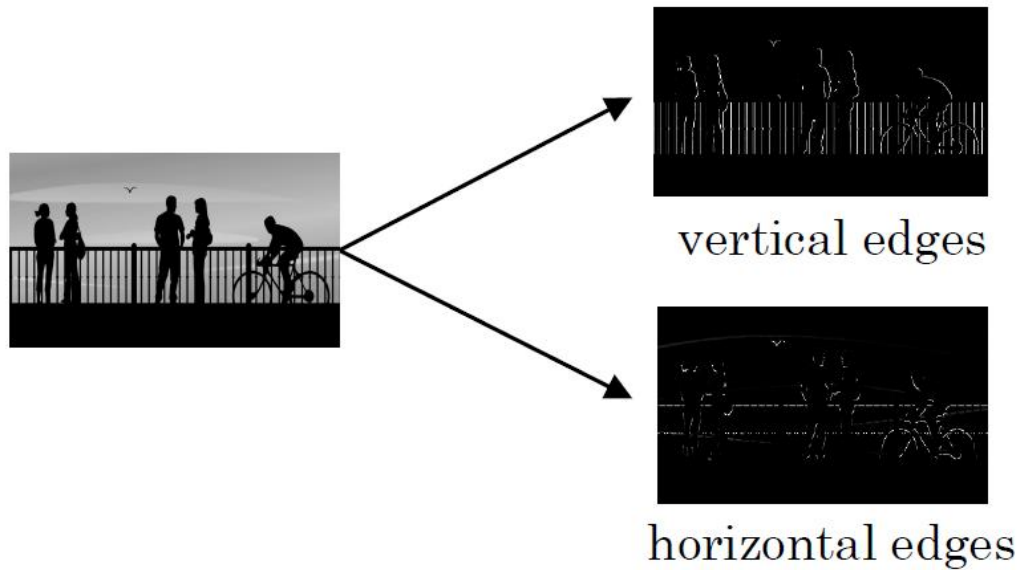
|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |



\*



# Convolution for image feature extraction



# Practice with linear filters



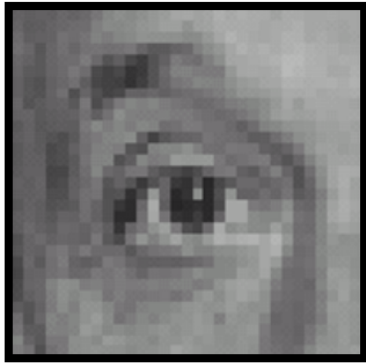
original

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |



Filtered  
(no change)

# Practice with linear filters

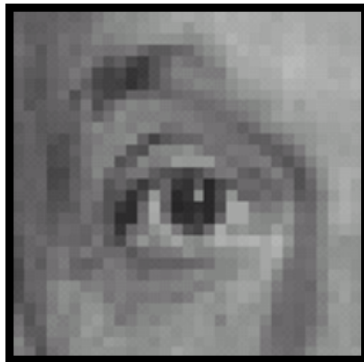


original

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 0 | 0 |

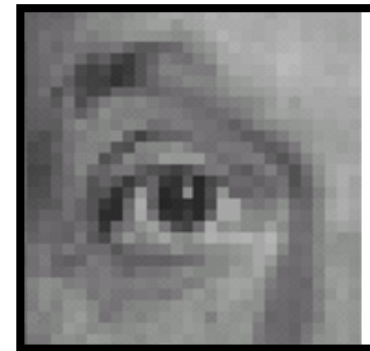
?

# Practice with linear filters



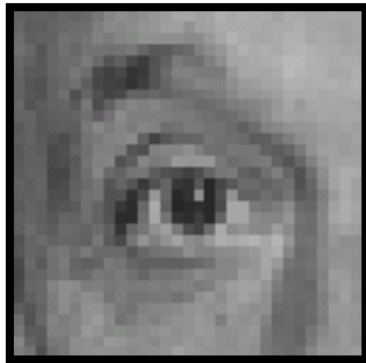
original

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 0 | 0 |



Shifted left  
By 1 pixel

# Practice with linear filters



original

$$\frac{1}{9}$$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

?



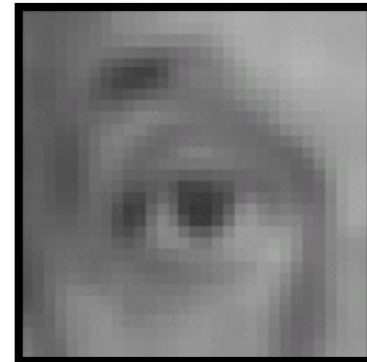
# Practice with linear filters



original

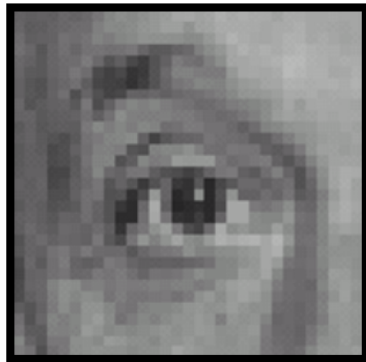
$$\frac{1}{9}$$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |



Blur (with a  
box filter)

# Practice with linear filters



original

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 2 | 0 |
| 0 | 0 | 0 |

-

$\frac{1}{9}$

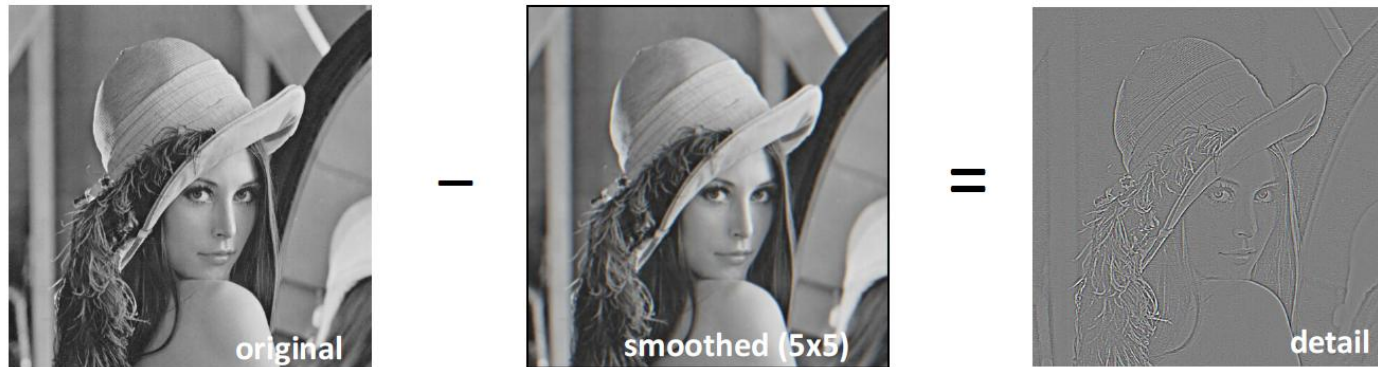
|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |



Sharpening filter

# Image sharpening

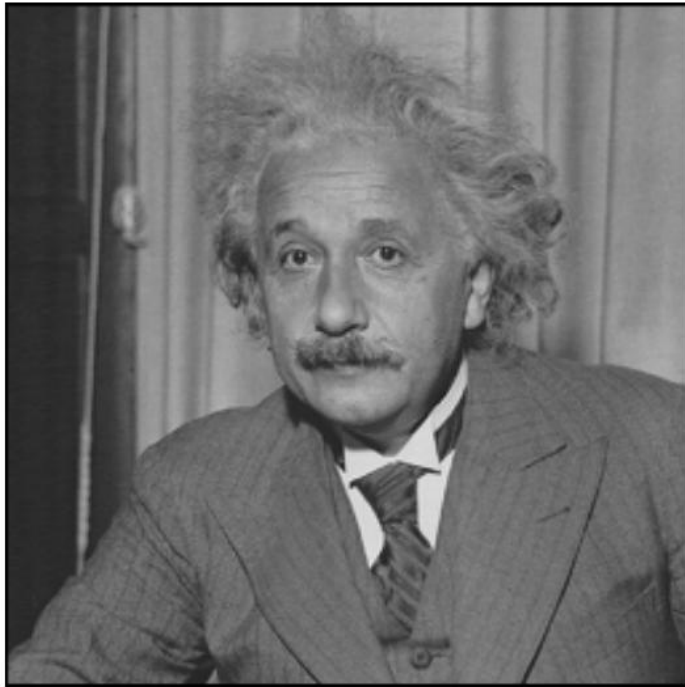
What does blurring take away?



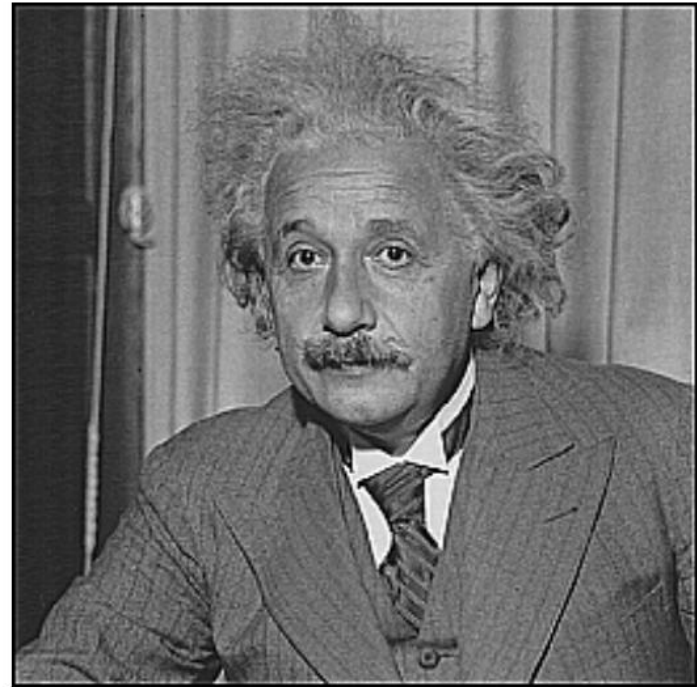
Let's add it back:



# Another example of image sharpening



**before**



**after**

# Object recognition

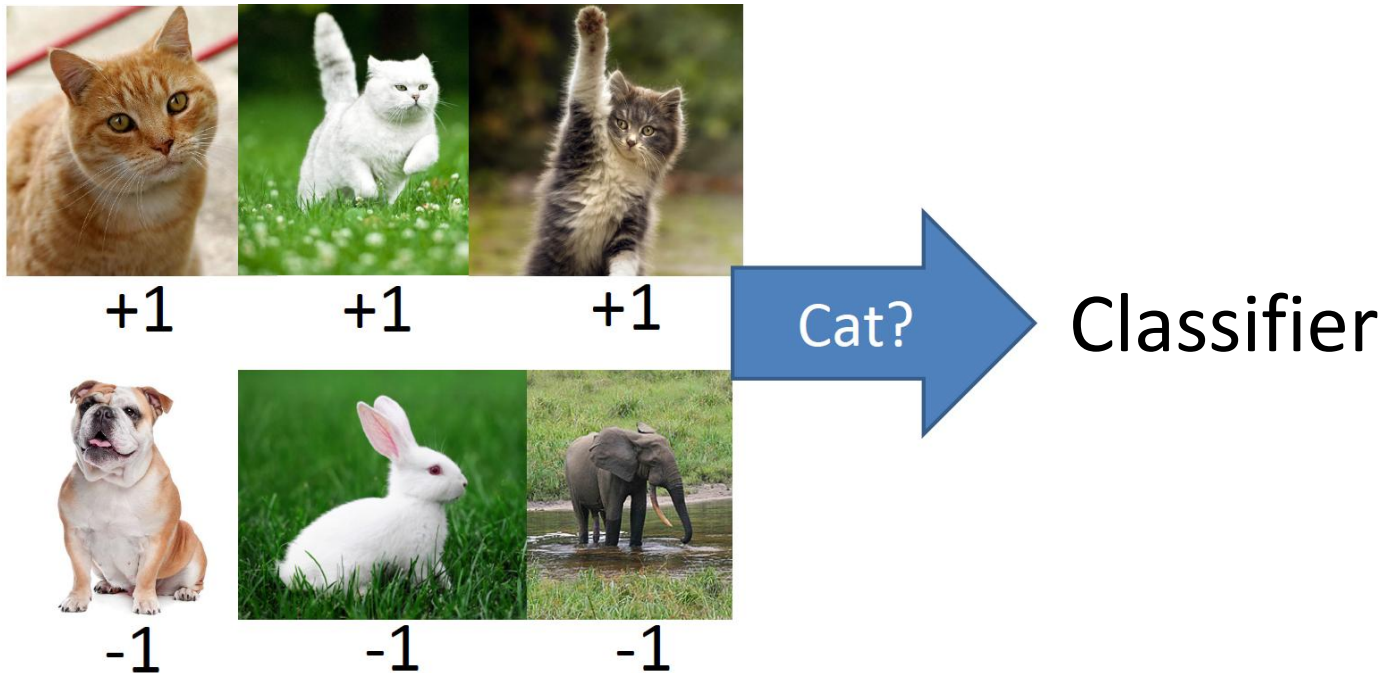


cat?



# Object recognition: training phase

- Collect a set of **training data** to learn a **classifier**





# Object recognition: testing phase

- Apply the learned **classifier** to the testing images



# Object recognition: testing phase

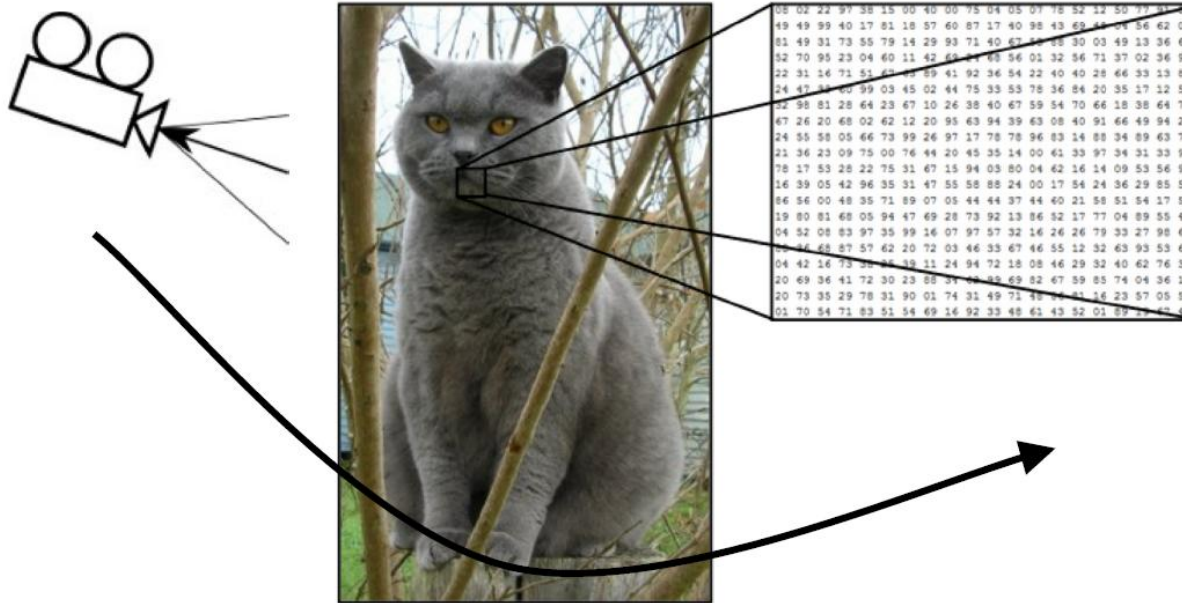
- Apply the learned **classifier** to the testing images and make **prediction**





# Why object recognition is so difficult

- Viewpoint variation



# Why object recognition is so difficult

- Illuminations



# Why object recognition is so difficult

- Deformations



# Why object recognition is so difficult

- Occlusions





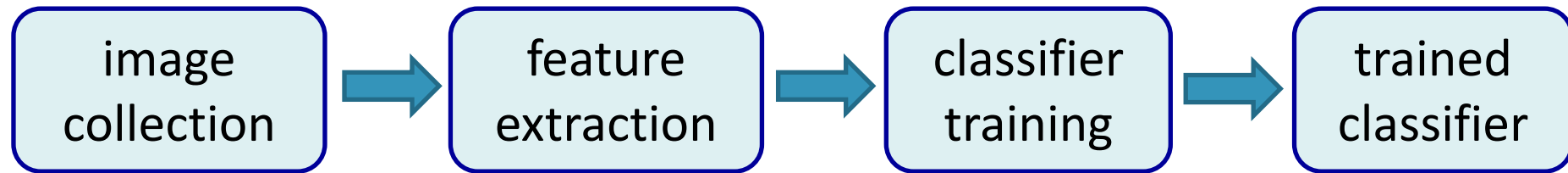
# Why object recognition is so difficult

- Background clutter

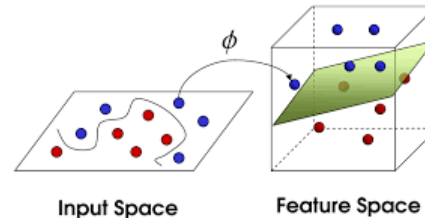


# Conventional approach to object recognition

- Training phase



histogram of oriented gradients (HoG)

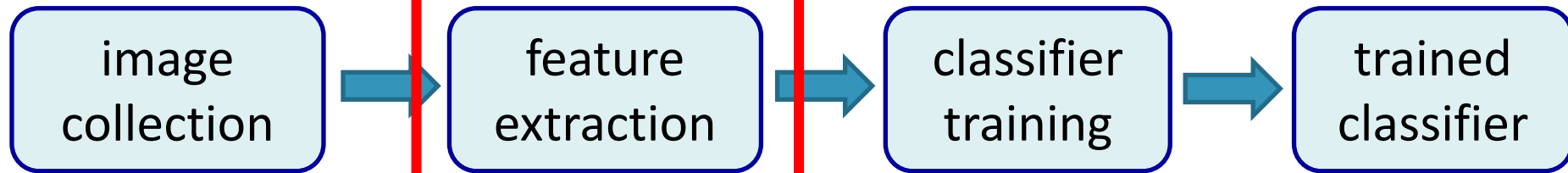


support vector machines (SVMs)

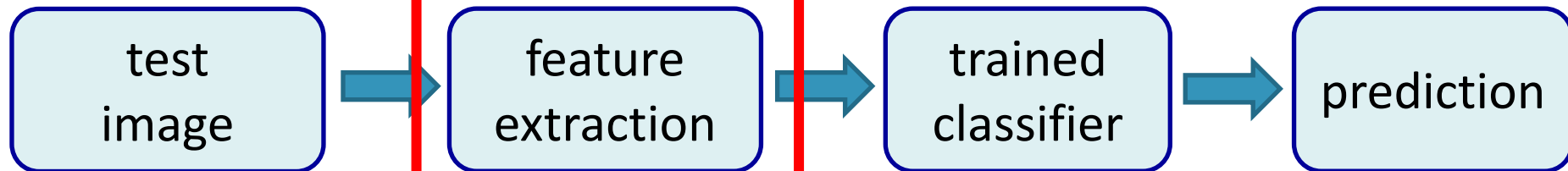
- dogs? ✖
- flowers? ○
- trains? ✖
- persons? ✖

# Conventional approach to object recognition

- Training phase



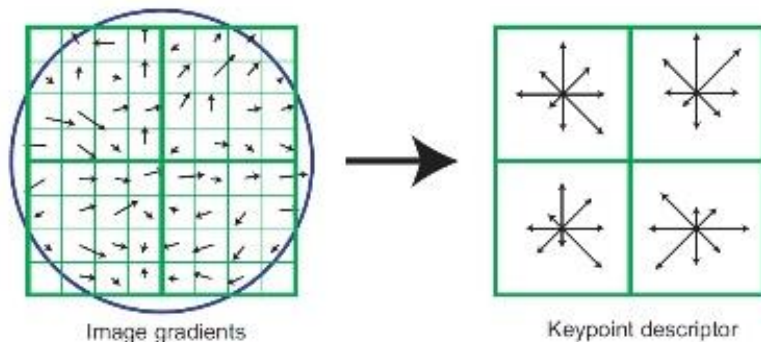
- Testing phase



application

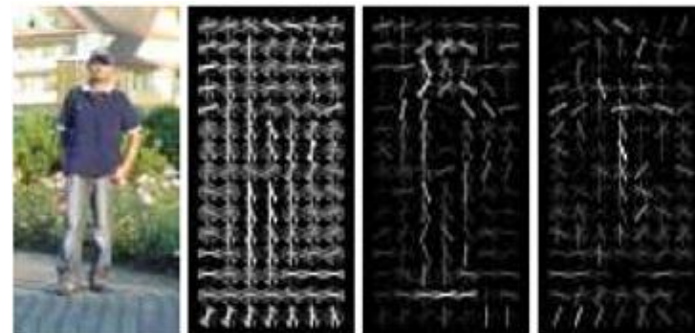
# Features are the keys

- Off-the-shelf visual features



SIFT [Lowe, IJCV'04]

Citations: 43465



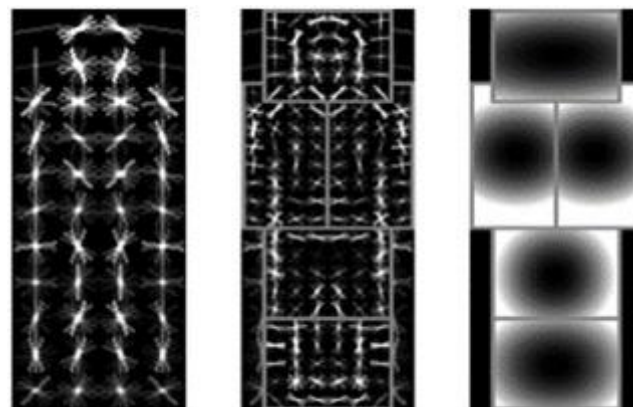
HoG [Dalal & Triggs, CVPR'05]

Citations: 20174



Constellation model [Fergus et al., CVPR'03]

Citations: 2551



DPM [Felzenszwalb et al., PAMI'10]

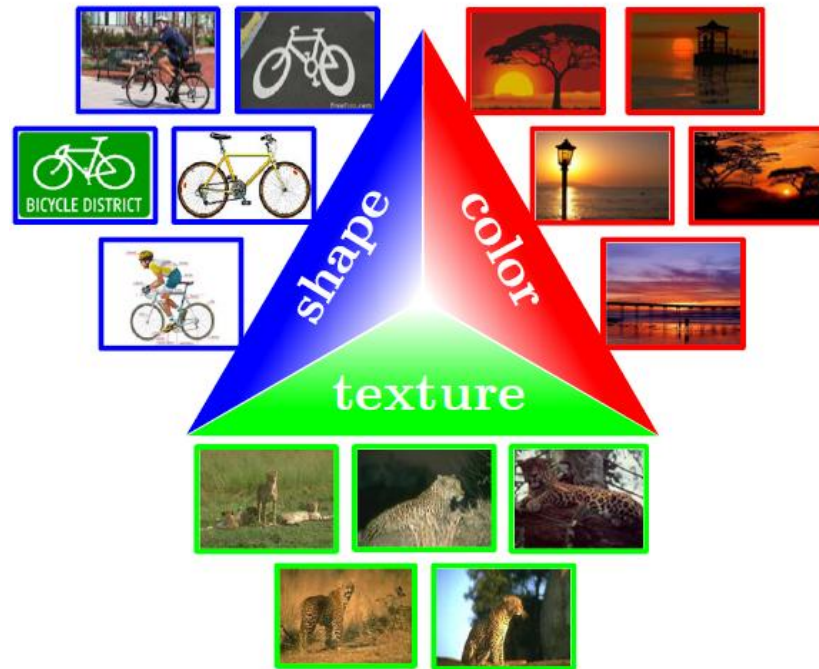
Citations: 5093





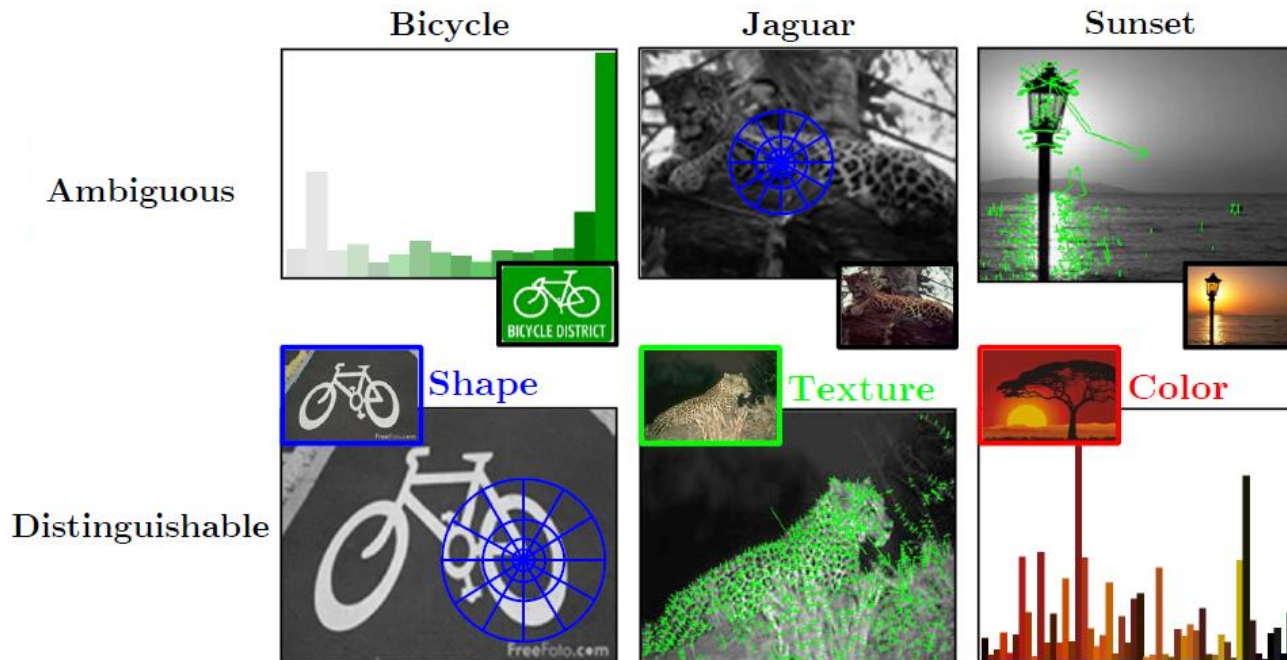
# Features are the key

- We extract features to represent an image
- Are handcrafted features optimal?



# Features are the key

- The optimal features for classification vary from category to category



# Conventional approaches vs. Deep learning

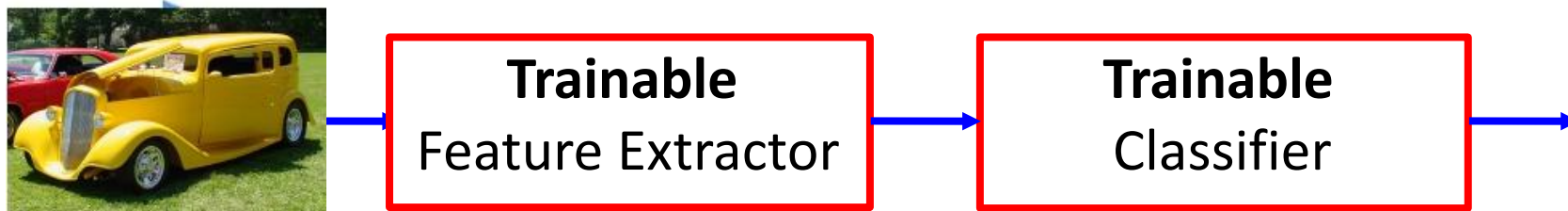
- Conventional approaches

- **Fixed/engineered** features + trainable classifier



- Deep learning / End-to-end learning / Feature learning

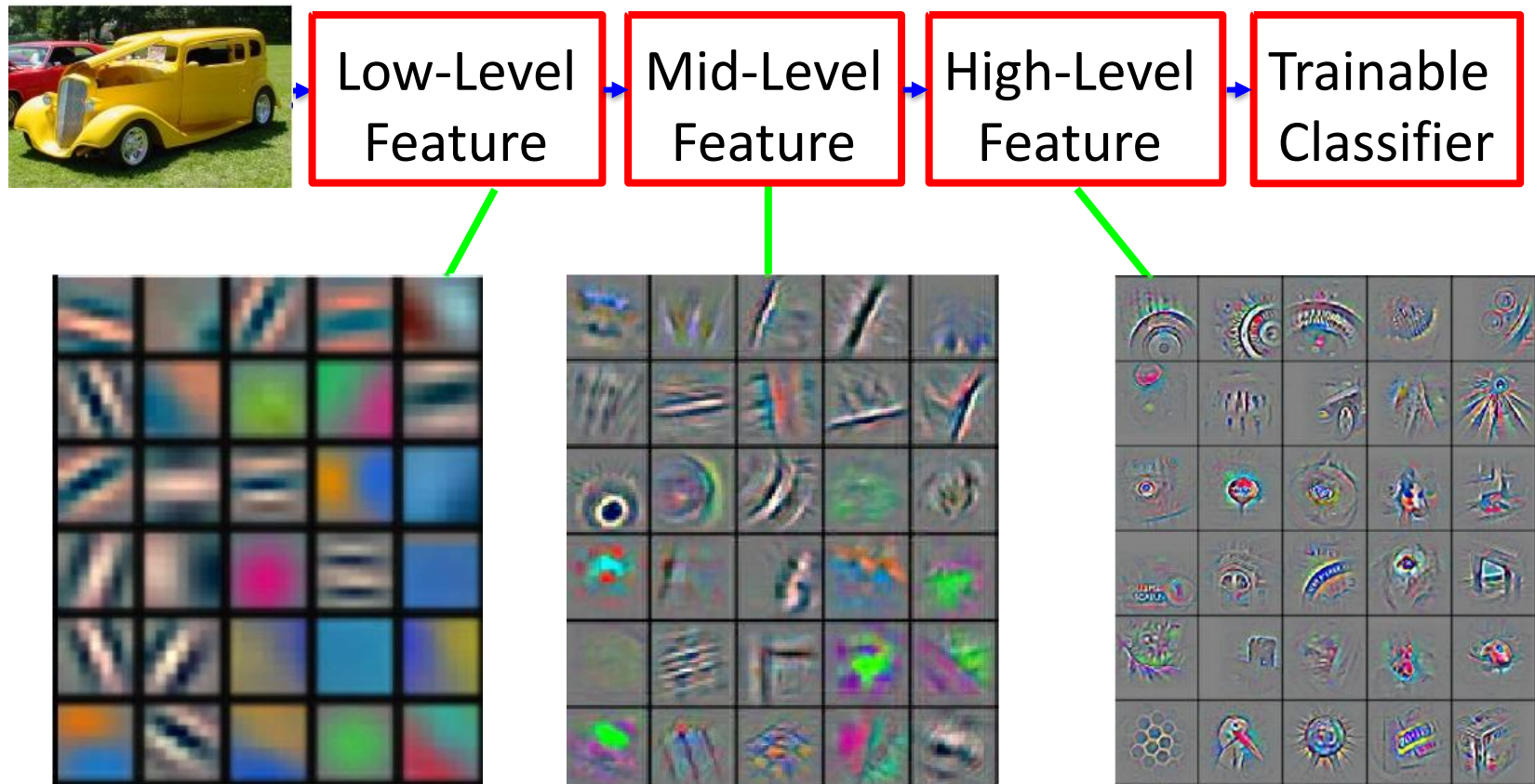
- **Trainable** features + trainable classifier



slide: Y LeCun & MA Ranzato



# Deep learning = Learning hierarchical representations

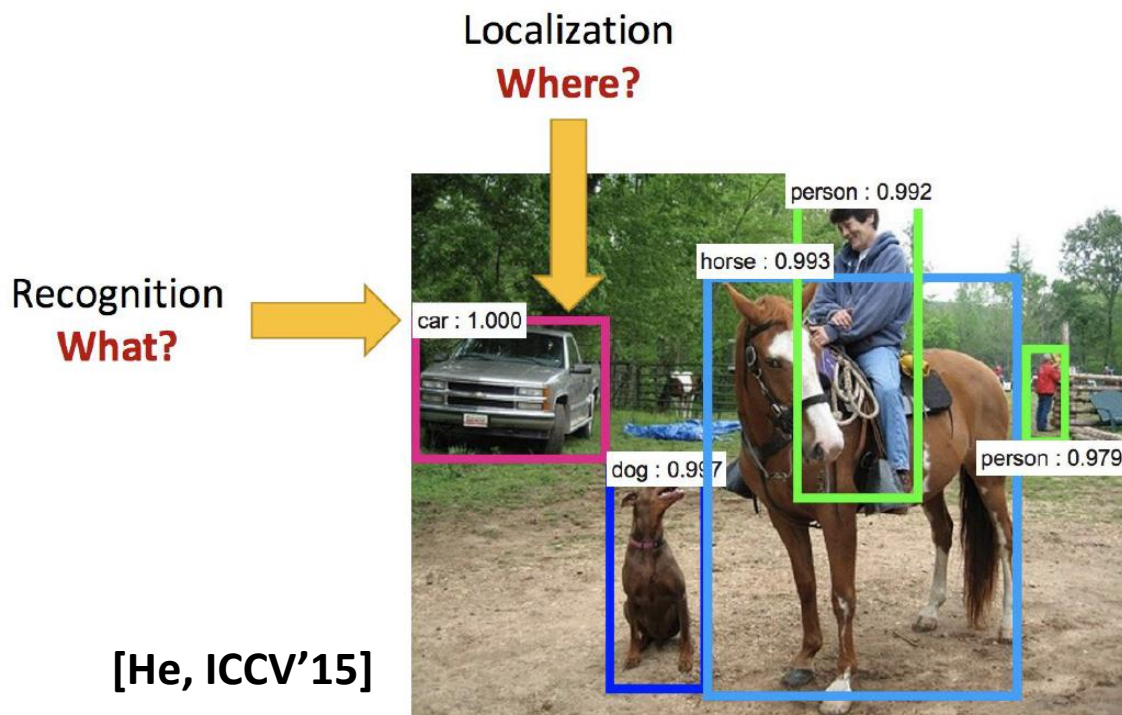


slide: Y LeCun & MA Ranzato



# Object detection

- Goal: Detecting instances of semantic objects of certain classes
- Critical to high-level vision tasks such as surveillance, self-driving car, and image retrieval

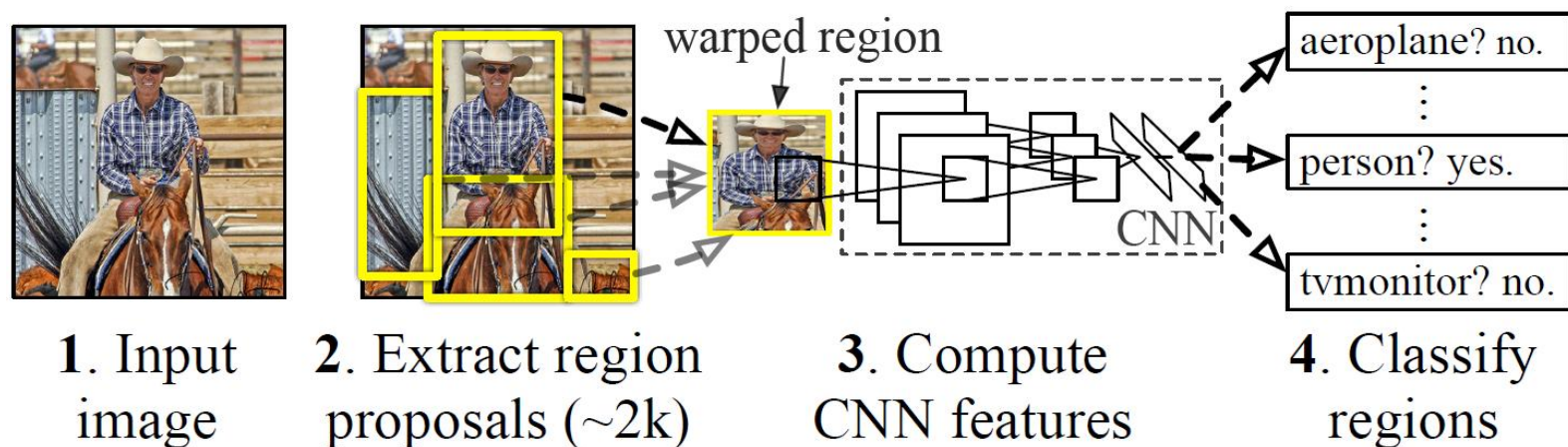




# R-CNN: Regions with CNN Features

[Girshick et al., CVPR'14]

- System overview

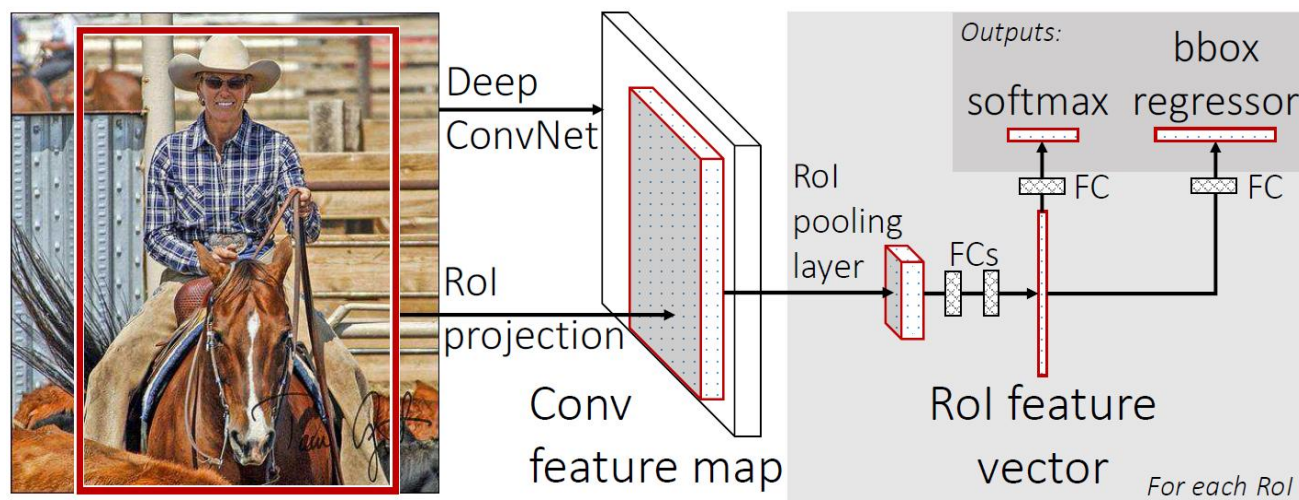


- Proposal extraction:** Using selective search [Uijlin et al., IJCV'13]
- Compute CNN features in the layer 'fc7' of Caffe CNN
- Region classification: linear SVMs or a softmax classifier
- Regression-based bounding box refinement

# Fast R-CNN

[Girshick, ICCV'15]

- System overview



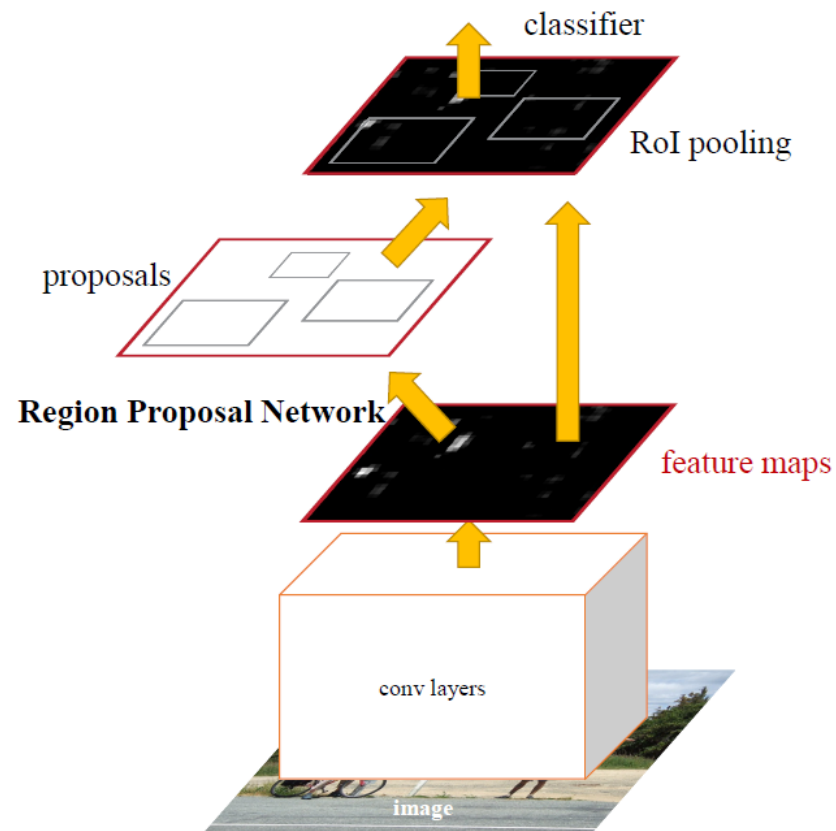
- Apply **fully convolutional networks** to the whole image
- RoI pooling**: each proposal is pooled into a fix-size feature map
- Classification with a softmax layer
- Regression-based bounding box refinement



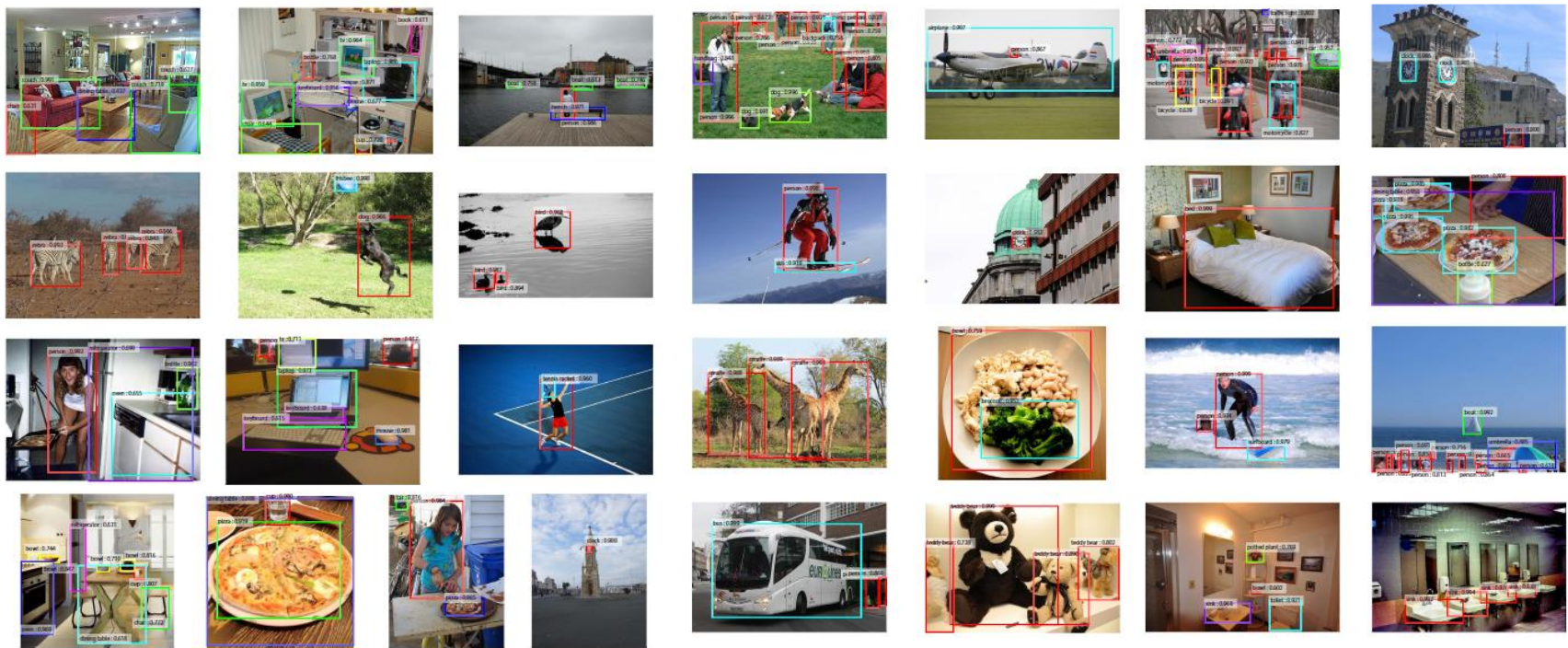
# Faster R-CNN

[Ren et al., NIPS'15]

- Selective search is removed
- Region proposal network (RPN)
  - After the last conv. Layer
  - Produces proposals
- GPU acceleration is allowed



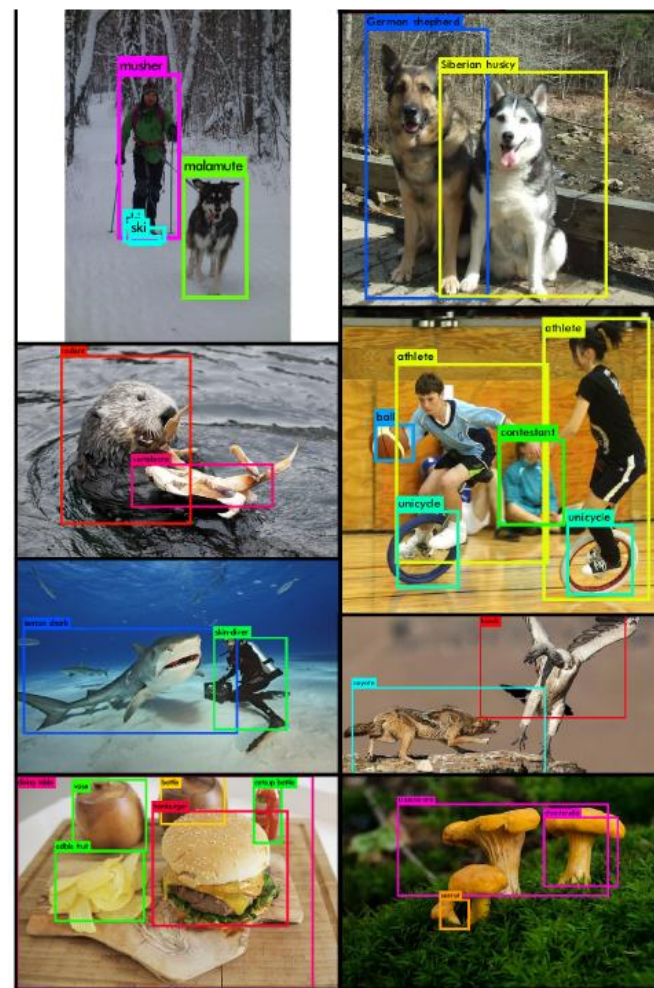
# Detection results by Faster R-CNN



# YOLO9000

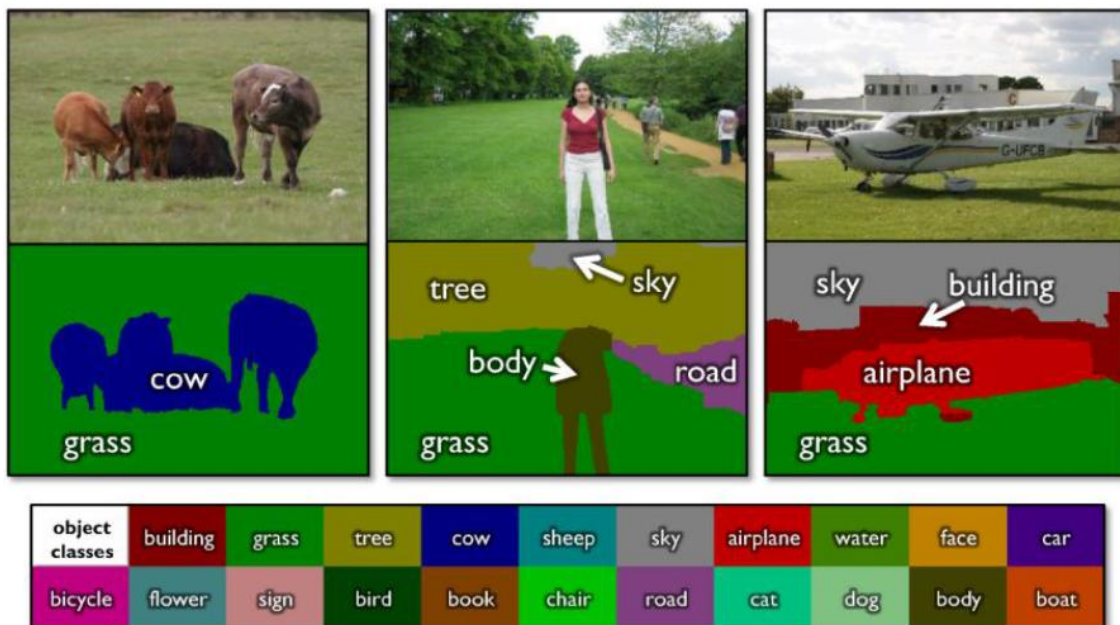
[Redmon & Farhadi, CVPR'17]

| Detection Frameworks    | Train     | mAP         | FPS |
|-------------------------|-----------|-------------|-----|
| Fast R-CNN [5]          | 2007+2012 | 70.0        | 0.5 |
| Faster R-CNN VGG-16[15] | 2007+2012 | 73.2        | 7   |
| Faster R-CNN ResNet[6]  | 2007+2012 | 76.4        | 5   |
| YOLO [14]               | 2007+2012 | 63.4        | 45  |
| SSD300 [11]             | 2007+2012 | 74.3        | 46  |
| SSD500 [11]             | 2007+2012 | 76.8        | 19  |
| YOLOv2 $288 \times 288$ | 2007+2012 | 69.0        | 91  |
| YOLOv2 $352 \times 352$ | 2007+2012 | 73.7        | 81  |
| YOLOv2 $416 \times 416$ | 2007+2012 | 76.8        | 67  |
| YOLOv2 $480 \times 480$ | 2007+2012 | 77.8        | 59  |
| YOLOv2 $544 \times 544$ | 2007+2012 | <b>78.6</b> | 40  |



# Semantic segmentation

- Goal: Label each pixel to one of predefined classes (or background)
- Critical to high-level vision tasks such as scene understanding, robot navigation, and image retrieval



[Shotton et al., 2007]





# Fully convolutional networks for semantic segmentation (FCN)

[Long et al. CVPR'15]

- Fully convolutional networks
  - There are only convolution layers and transposed convolution layers in the network architecture.
- Modify convolution nets for classification
  - The networks were pre-trained on other datasets such as ImageNet for classification and then fine-tuned with datasets for semantic segmentation such as Pascal VOC.



input image from Pascal VOC 2012

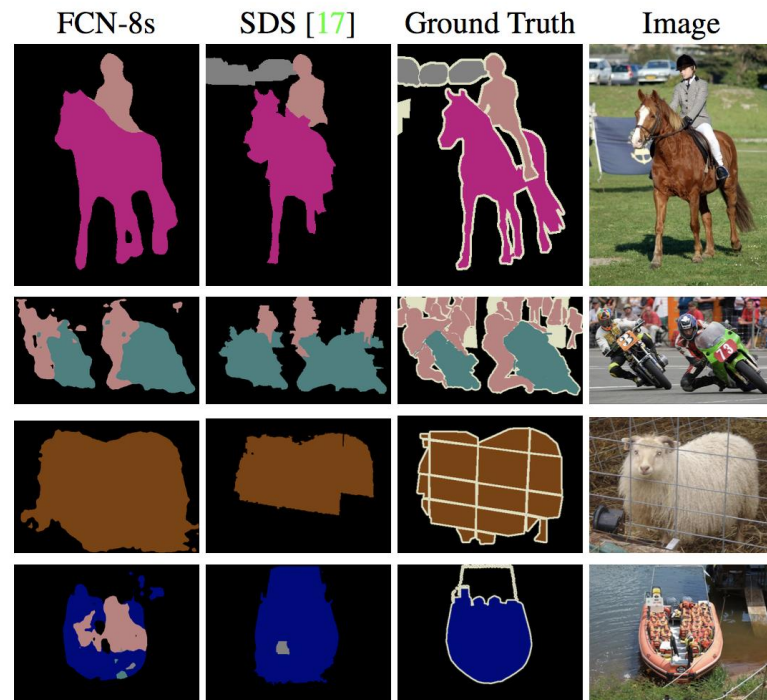


segmentation results

# Experimental Results

- Dataset : Pascal VOC
  - Training : [Hariharan *et al*] collected labels for a larger set of 8498 PASCAL training images .
  - Testing : Test set of Pascal VOC 2011 and 2012

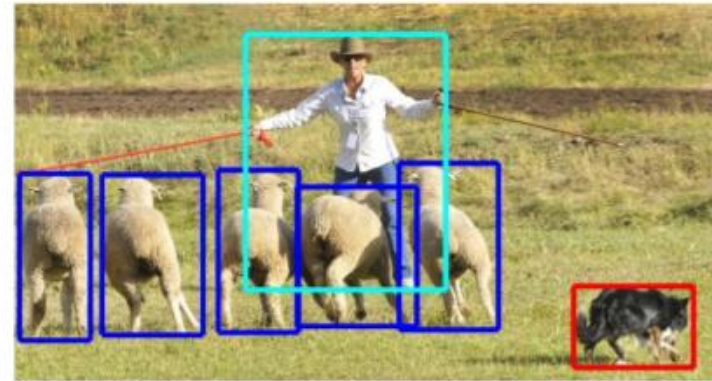
|            | mean IU<br>VOC2011 test | mean IU<br>VOC2012 test | inference<br>time |
|------------|-------------------------|-------------------------|-------------------|
| R-CNN [12] | 47.9                    | -                       | -                 |
| SDS [17]   | 52.6                    | 51.6                    | ~ 50 s            |
| FCN-8s     | <b>62.7</b>             | <b>62.2</b>             | ~ <b>175 ms</b>   |



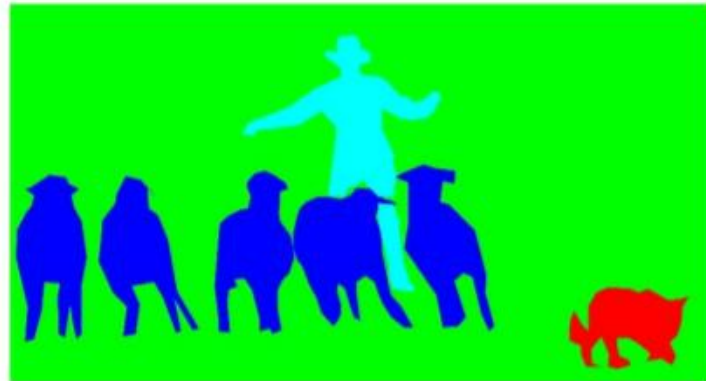
# Object instance segmentation



(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) Instance segmentation



# Instance segmentation

[He et al. ICCV'17]

- Instance segmentation: **Instance-aware** semantic segmentation
  - Instance-specific detection
  - Pixel-wise segmentation
- **Mask-RCNN**
  - Object detection + Semantic segmentation
  - Faster R-CNN + FCN



semantic segmentation



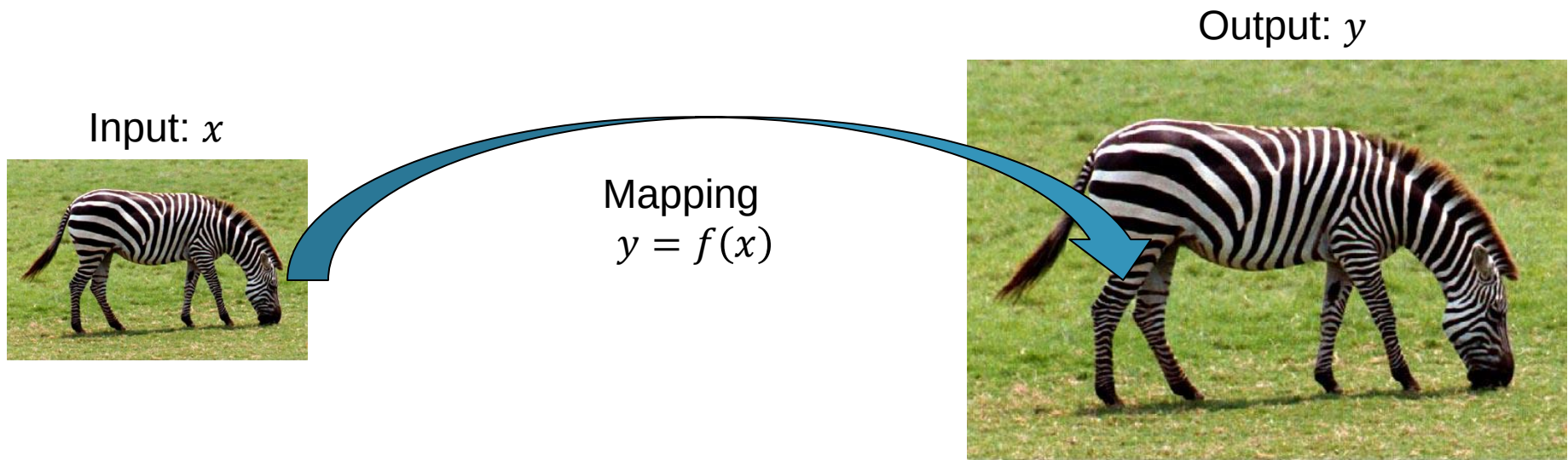
instance segmentation

# Instance segmentation by Mask R-CNN



# Image super-resolution

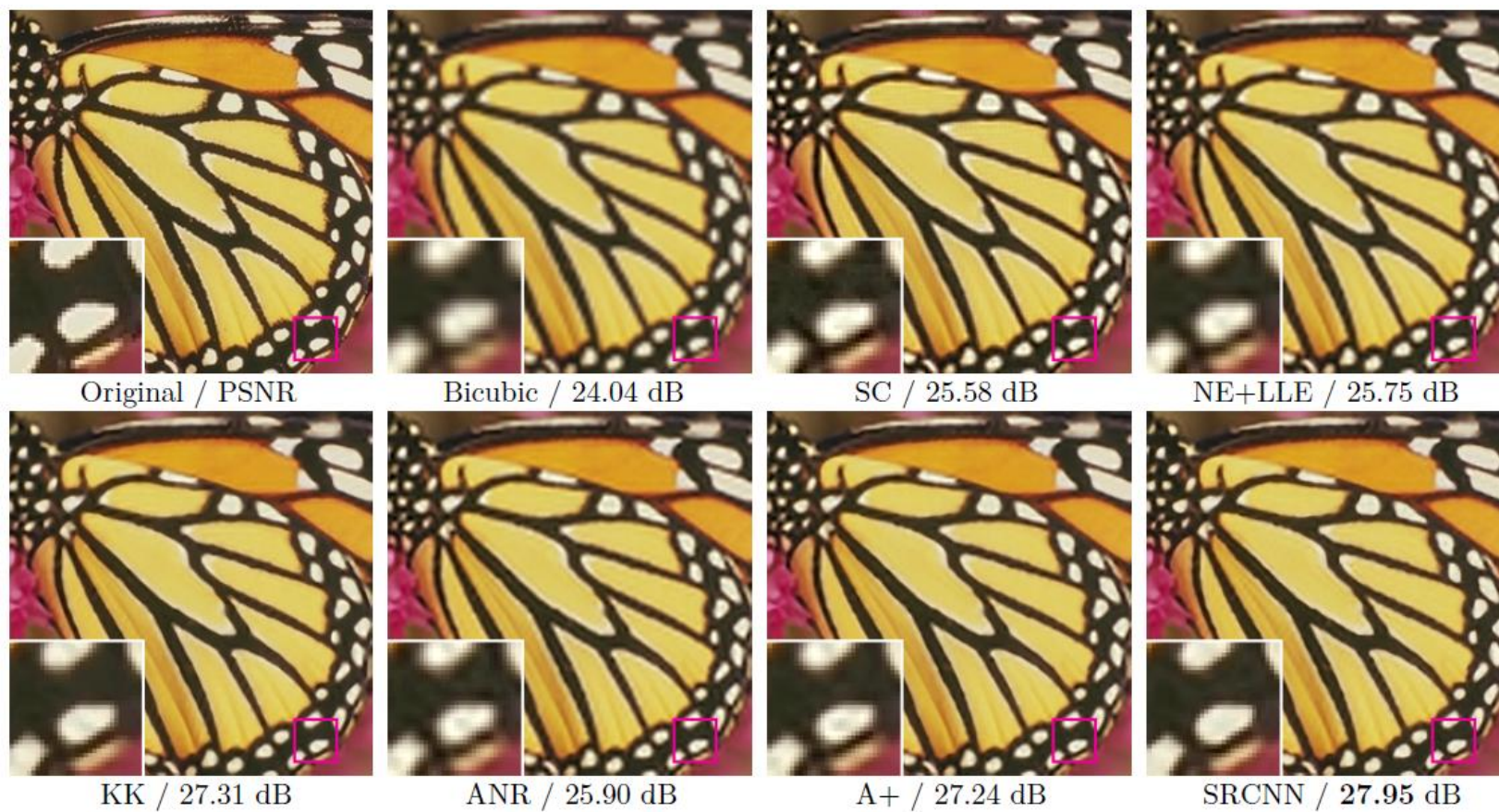
- Image super-resolution
  - Input: low-resolution (LR) images
  - Output: high-resolution (HR) images
  - Training goal: Learn the mapping between the inputs and outputs
  - Testing goal: Estimate the HR counterpart of a give LR image





# Experimental Results of SRCNN

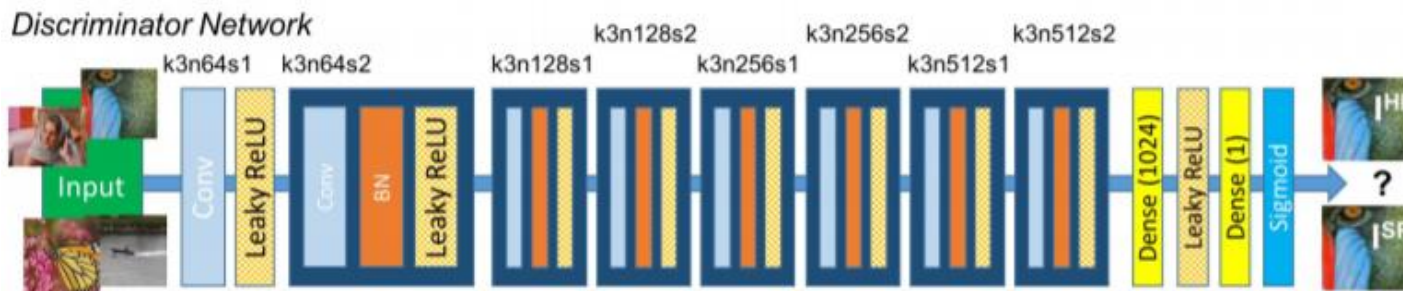
[Dong et al. ECCV'14]



# Super-resolution using generative adversarial nets (SRGAN)

[Ledig et al. CVPR'17]

- Previous work
  - Regard CNNs as the mapping function or generator
- Limitation:
  - Over-smoothing or blurring
  - Hard to recover the highly textured details
- **SRGAN: generator + discriminator**
  - Jointly learn the generator and discriminator
  - Make the results look more like the real-world images





# Experimental results

SRResNet: the state-of-the-art method but without the discriminator network

bicubic



SRResNet



SRGAN



original





# Image style transfer

[Gaty et al. CVPR'16]

- Given a **content image** and a **style image**, generate a new image that
  - Matches the **CNN features** of the content image
  - Matches the **Gram matrices** of the style image



content image

+



style image

=



stylized image

# Transfer results





# Action recognition

- Predict the behavior of a person in a **video** sequence



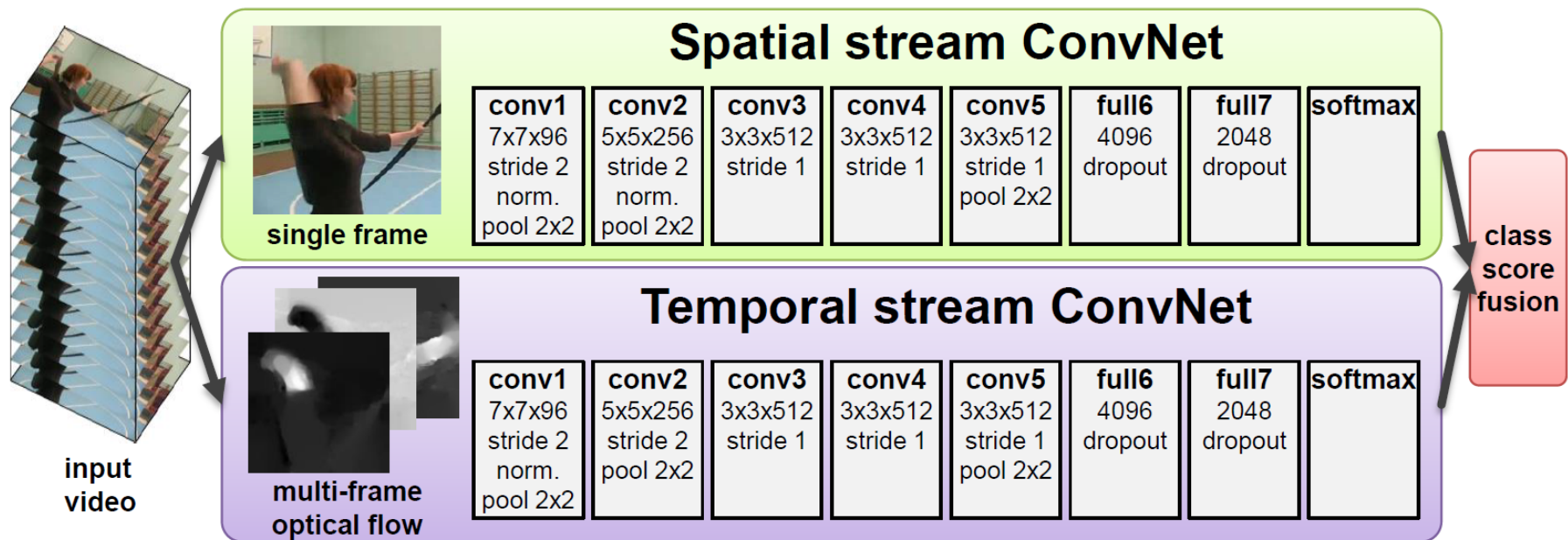
UCF Sports 101 Dataset



# Two-Stream CNNs for action recognition

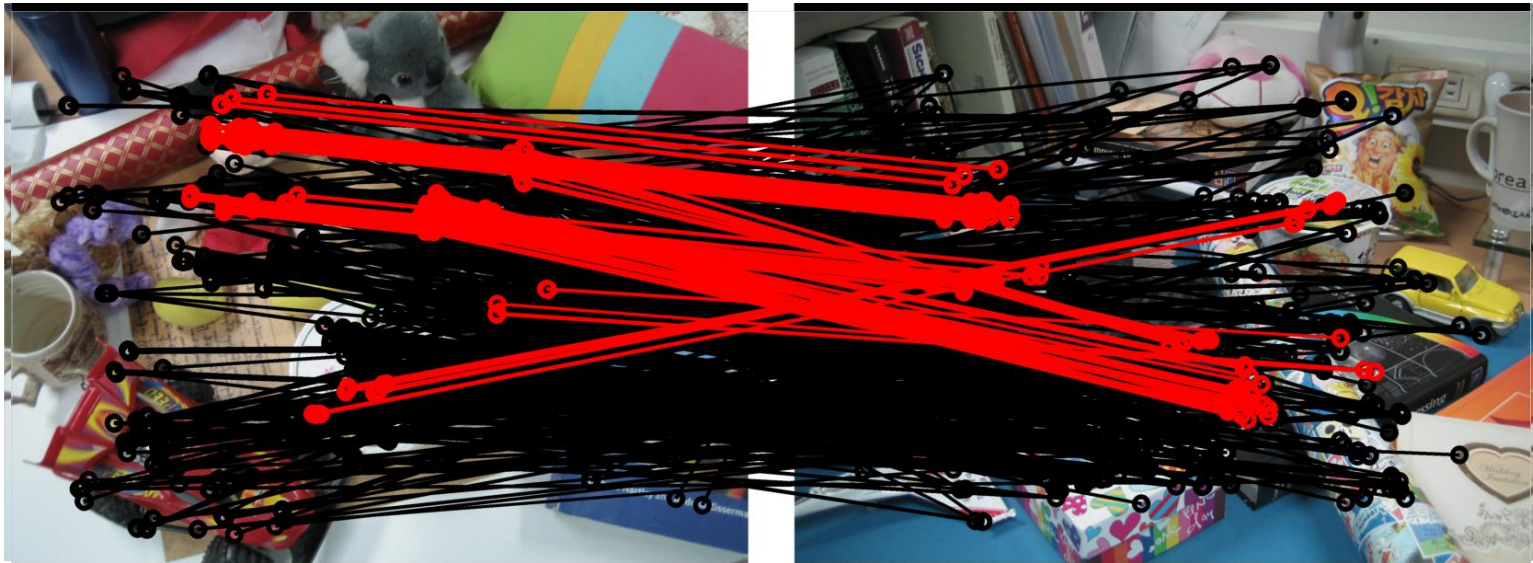
[Simonyan and Zisserman, NIPS'14]

- A two-stream CNN architecture
  - Spatial stream networks: Single-frame RGB image
  - Temporal stream networks: Multi-frame dense optical flow



# Image matching

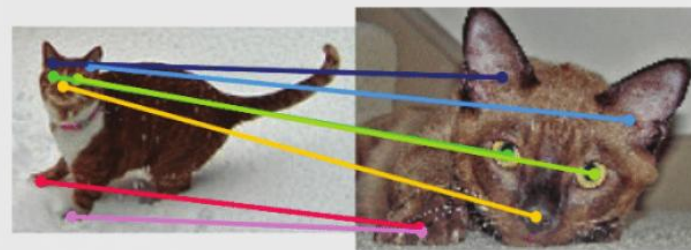
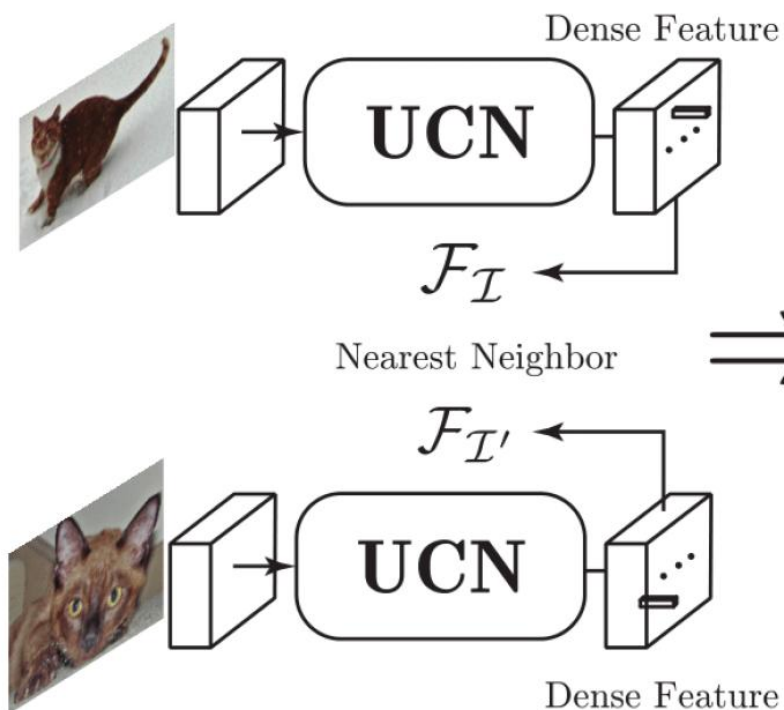
- Seek common objects and regions between images
- Conventional pipeline:
  - Salient point detection + Description + Matching



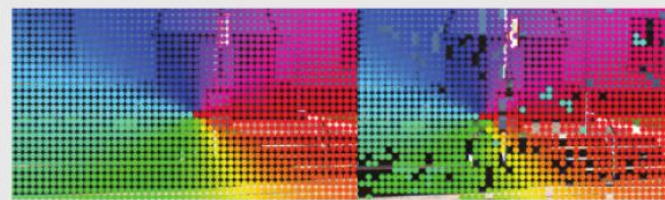


# Universal correspondence network (UCN)

[Choy et al, NIPS'16]



Accurate semantic correspondences

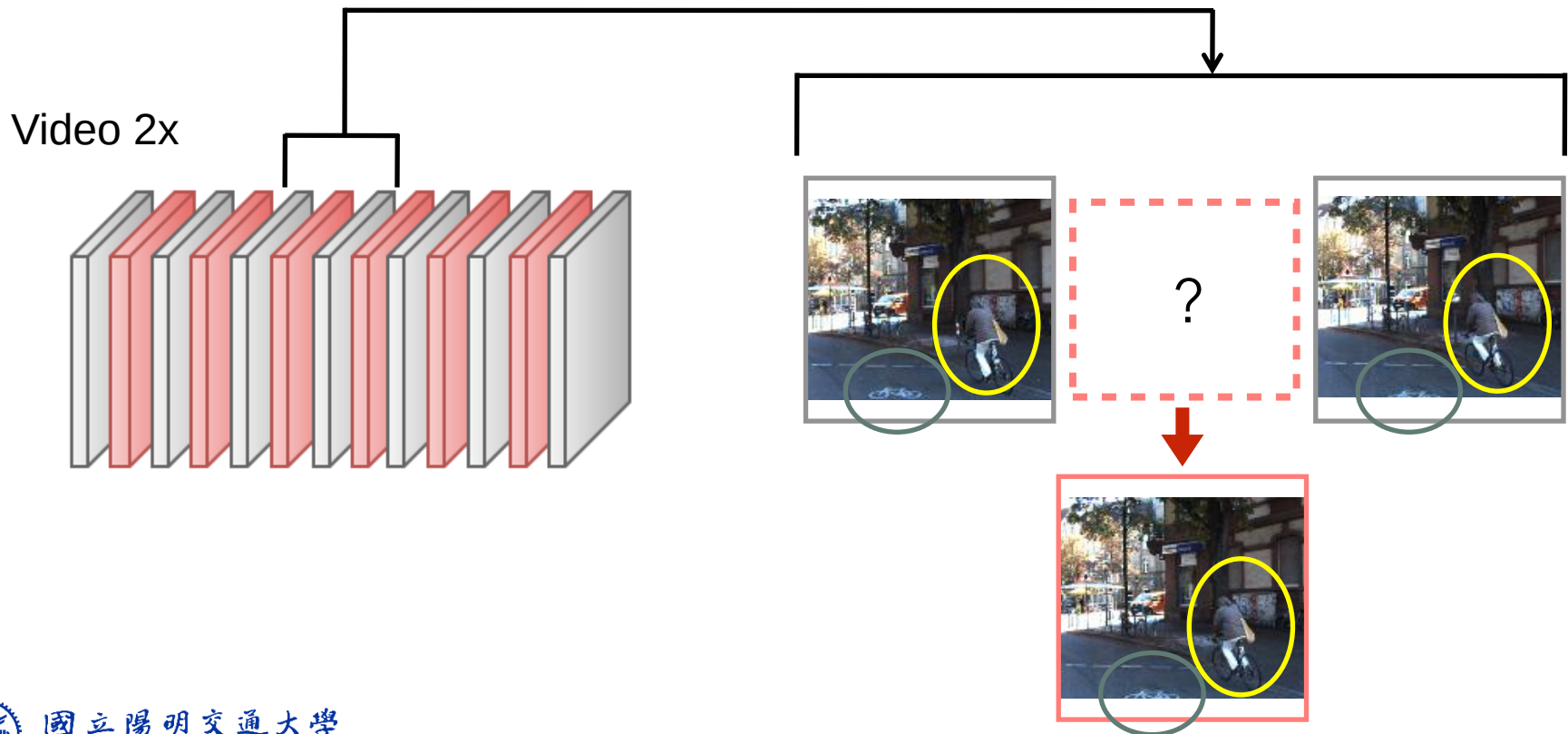


Dense accurate geometric correspondences

- Convolutional spatial transformer
- Fully convolutional
- Learns feature directly
- Active hard negative mining

# Video frame interpolation

- Video interpolation produces videos of higher frame rates
  - Problem formulation: Predict the intermediate frame between two consecutive frames



# Video frame interpolation

[Liu et al. AAAI'19]

Original Video



8X Video



# Video frame interpolation

[Liu et al. AAAI'19]



1X



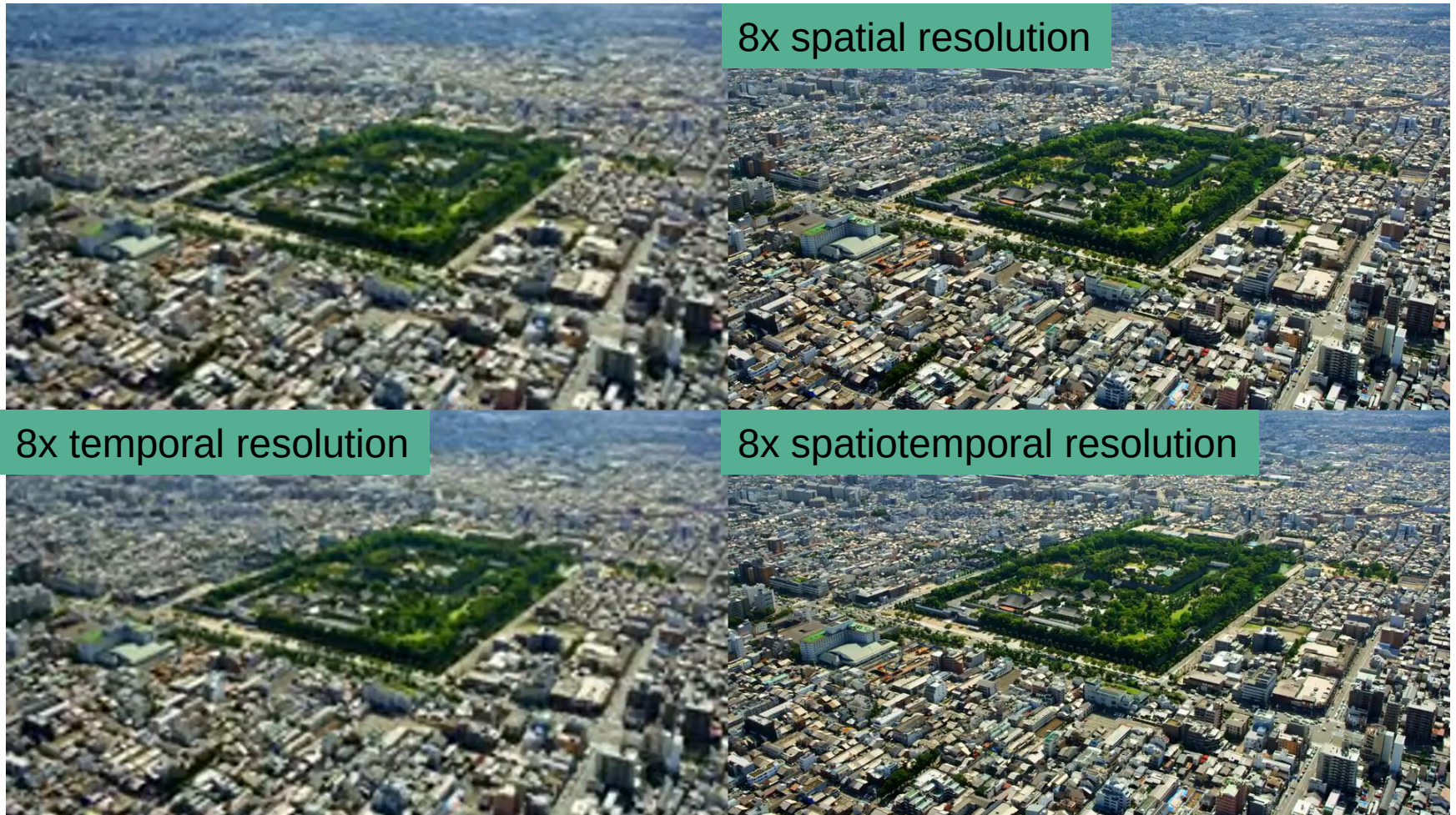
8X





# Spatiotemporal super-resolution

[Lin et al. IJCAI'20]



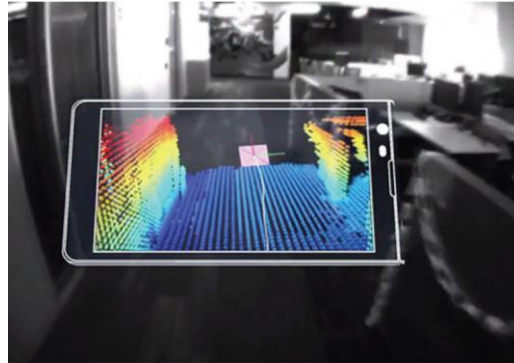


# 3D points classification and segmentation

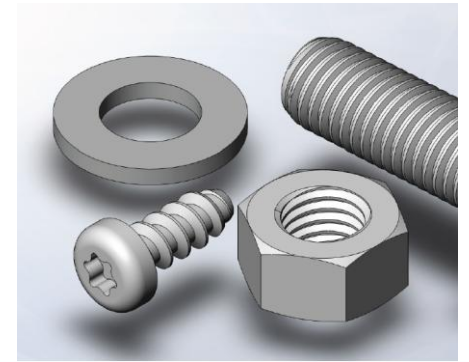
- Emerging 3D computer vision applications



3D robot perception



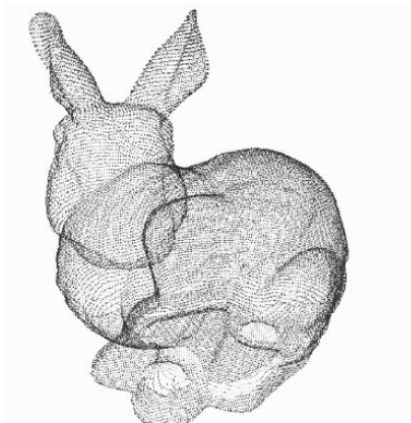
augmented reality



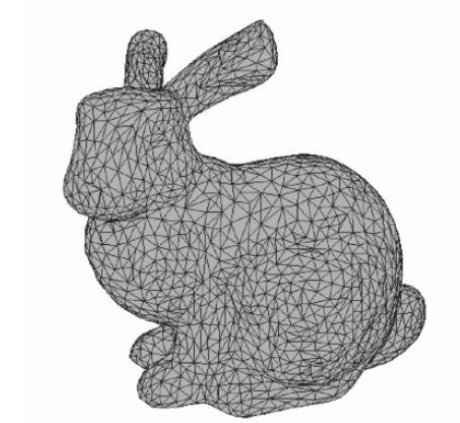
shape design

# 3D data representations

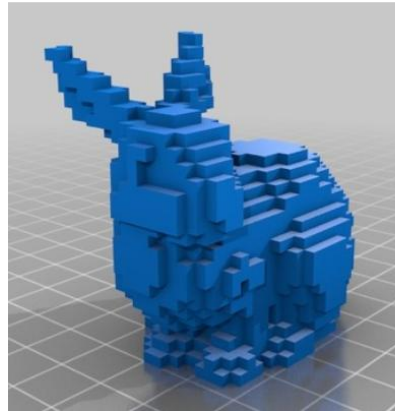
- Point cloud: A collection of 3D points
- Mesh: A collection of triangle faces
- Voxel: A 3D volumetric grid



point cloud



mesh



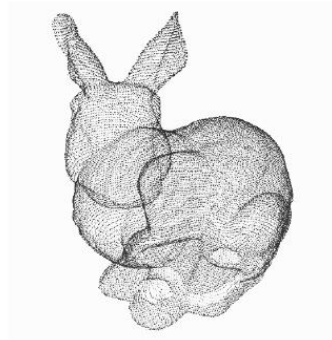
volumetric



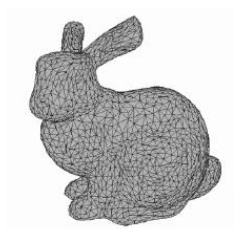
multiple projected  
RGBD images

# Why 3D point clouds?

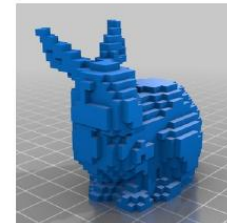
- A 3D point cloud is a collection of 3D points, each of which is represented by its 3D coordinate  $(x, y, z)$
- Point cloud is close to raw sensing data
- Point cloud is canonical



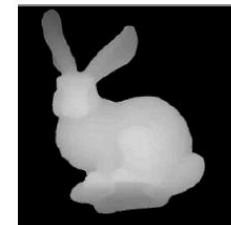
**Point Cloud**



Mesh



Volumetric

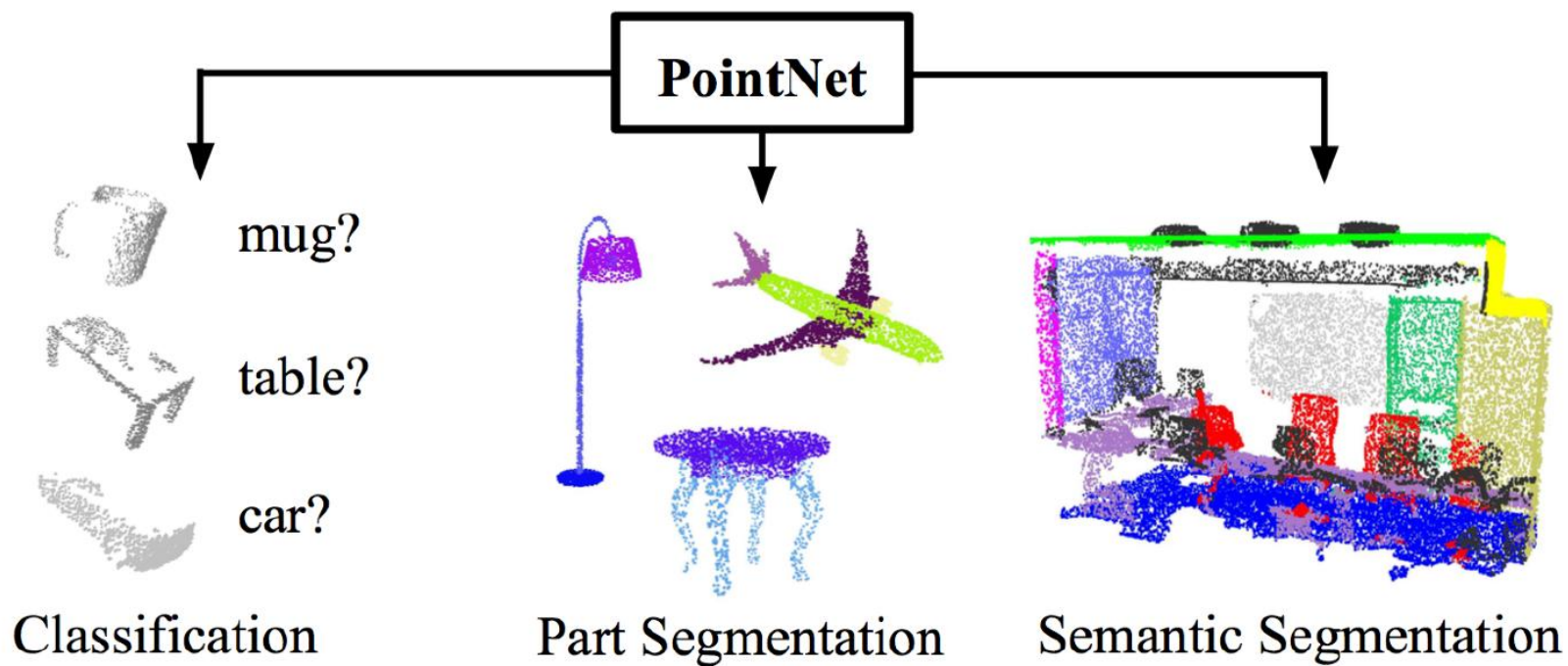


Depth Map

# PointNet

[Qi et al., CVPR'17]

- PointNet: It is an end-to-end learnable deep learning framework for scattered, unordered 3D point data



# NeRF 3D reconstruction

[Mildenhall et al., ECCV'20]

- Input: A set of images
- Output: 3D Representation



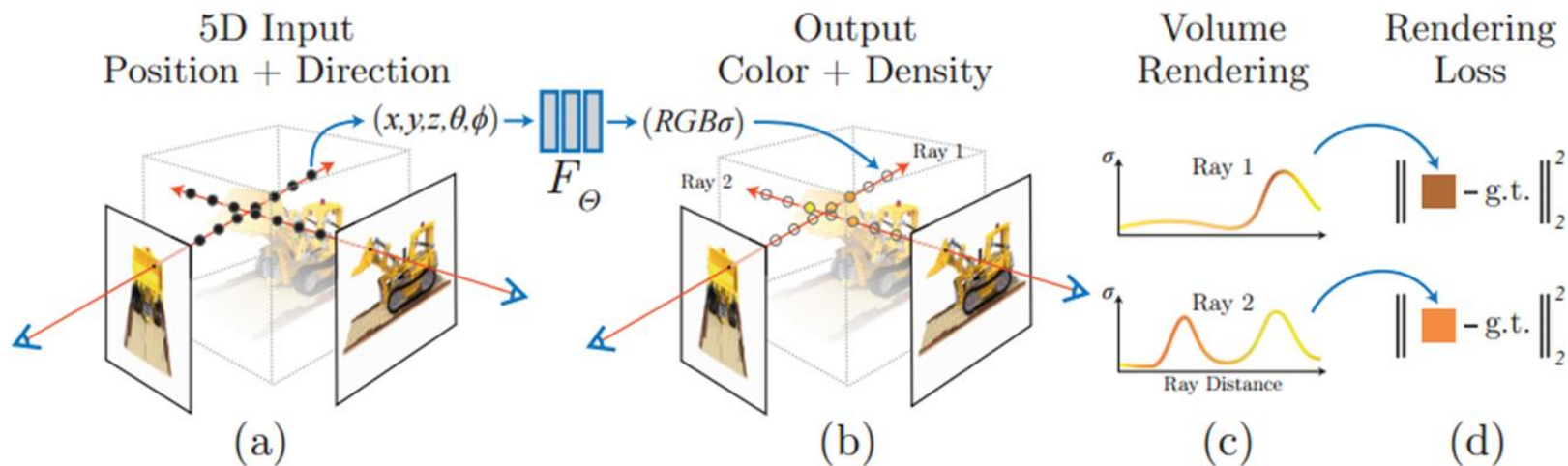
[NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis]





# NeRF overview

- a. Sample **5D coordinates** along camera rays
- b. Predict **RGB and density** by the model
- c. Using **volume rendering** to composite the values in the image
- d. Calculate the loss between prediction and ground truth
- Optimize the model by gradient descent



# Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

Yen-Yu Lin (林彥宇)

Email: [lin@cs.nycu.edu.tw](mailto:lin@cs.nycu.edu.tw)

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

